LIGHTWEIGHT REAL-TIME DETECTION OF SPEED BUMPS AND POTHOLES

A Project Report

U23ADS705 – MINI PROJECT

Submitted by

BENADICT INFANT A (61782323110022)

ELANGOVAN S (61782323110035)

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND DATA SCIENCESONA COLLEGE OF TECHNOLOGY (AUTONOMOUS), SALEM – 636005

ANNA UNIVERSITY: CHENNAI 600 025

OCTOBER 2026

SONA COLLEGE OF TECHNOLOGY (AUTONOMOUS)
ANNA UNIVERSITY: CHENNAI – 600 025

BONAFIDE CERTIFICATE

Certified that this project titled "GV-YOLO26: LIGHTWEIGHT REAL-TIME DETECTION OF SPEED BUMPS AND POTHOLES" is the bonafide work of "BENADICT INFANT A (61782323110022), ELANGOVAN S (61782323110035)" who carried out the project work U23ADS705 – MINI PROJECT under my supervision.

Signature of the Supervisor

Dr. P. SHANMUGARAJA
SUPERVISOR
Assistant Professor,
Department of Information Technology,
Sona College of Technology,
Salem – 636005.

Head of the Department

Dr. J. AKILANDESWARI
HEAD OF THE DEPARTMENT
Department of Information Technology,
Sona College of Technology,
Salem – 636005.

Submitted for the Project Viva-Voce Examination held on _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

Sona College of Technology

Department of Information Technology

U23ADS705 – MINI PROJECT – DECLARATION FORM

Declaration by the Student(s)

We hereby declare that the project titled "GV-YOLO26: Lightweight Real-Time Detection of Speed Bumps and Potholes" is an original work carried out by us under the guidance of Dr. P. Shanmugaraja. This project has not been submitted elsewhere for the award of any degree.

We also affirm that all the data and information used in this project are genuine and properly acknowledged. This project work is carried out with reference to the United Nations Sustainable Development Goals **Goal 3 – Good Health and Well-being, Goal 9 – Industry, Innovation and Infrastructure, and Goal 11 – Sustainable Cities and Communities**.

S.No	Register Number	Name	Signature
1.	61782323110022	BENADICT INFANT A	
2.	61782323110035	ELANGOVAN S	

Date: _____

Place: Sona College of Technology

ACKNOWLEDGMENT

We would like to place on record our sincere thanks to the Management of Sona College of Technology, Salem, for providing the facilities and the academic environment in which this project was carried out.

We are grateful to our Principal for the encouragement given to students to take up project work that goes beyond the classroom.

We thank **Dr. J. Akilandeswari**, Head of the Department of Information Technology, for supporting this work and for making the department's resources available to us whenever we needed them.

We express our deep gratitude to our project guide, **Dr. P. Shanmugaraja**, Assistant Professor, Department of Information Technology, for his guidance at every stage of this project. His questions about what each part of the model actually contributes pushed us to look past the headline numbers, and that shaped the final form of this work.

We also thank the project coordinators and all the faculty members of the department for their review comments, which helped us sharpen both the experiments and this report.

We also acknowledge the open-source Ultralytics framework and the contributors of the public Humps/Bumps and Potholes dataset, on which the models in this project were built and evaluated.

Finally, we thank our parents and friends for their patience and support throughout the project.

BENADICT INFANT A

ELANGOVAN S

ABSTRACT

Speed bumps and potholes are the road anomalies that most often force a vehicle to slow down or swerve. On Indian roads many speed bumps are unpainted and poorly marked, so a driver often gets no visual warning until the suspension takes the impact. A camera that detects these anomalies early could warn the driver or let an active suspension prepare in advance, but only if the detector is accurate on small, distant targets and fast enough to run on an embedded board.

This project proposes GV-YOLO26, a lightweight real-time detector for speed bumps and potholes built on YOLO26n, the newest end-to-end member of the YOLO family. We redesigned its backbone and neck around cheap convolutions: GhostConv layers replace the costly downsampling convolutions of the deeper stages, and VoVGSCSPC blocks built from GSConv replace the C3k2 feature blocks. A high-resolution P2 detection level is added for small, distant targets, and a Lightweight Grouped Detection Head (LGDH) with shared grouped convolutions keeps the extra level affordable while retaining the NMS-free one-to-one prediction of YOLO26. In its final form GV-YOLO26 has 1.57 million parameters and needs 5.1 GFLOPs, 34% fewer parameters and less computation than YOLO26n (2.38 million, 5.3 GFLOPs), even though it predicts on an extra 160 × 160 grid. Training incorporates a Normalized Wasserstein Distance (NWD) box loss and weather augmentation; knowledge distillation from a larger GV-YOLO26 teacher was also evaluated, and an in-video hazard detection pipeline with temporal tracking was demonstrated, with embedded hardware acceleration identified as future work.

To choose the base model on evidence, we first benchmarked seven stock detectors, from YOLOv8n to YOLO26n and RT-DETR-L, on the SBP-YOLO road hazard dataset; YOLO26n was the most accurate at 74.8% mAP@0.5 and the cheapest at 5.3 GFLOPs. GV-YOLO26 was then trained stage by stage through an ablation ladder on the 7,600-image SBP-YOLO dataset. We found that 47% of the dataset's raw test images are augmented copies or video frames of training scenes, and evaluated every model on a leakage-free held-out set of 482 images instead. There GV-YOLO26 reached 77.5% mAP@0.5 against 81.0% for an identically trained YOLO26n baseline, with 34% fewer parameters, running at 57.5 FPS in PyTorch FP32 on an NVIDIA Tesla T4 GPU. Distillation did not improve accuracy: as implemented, its box term was unbounded and destabilised training. An in-video demo links detections across frames with ByteTrack and warns only about hazards confirmed over three frames. TensorRT edge acceleration and embedded Jetson deployment are formulated as future work.

Keywords: GV-YOLO26, YOLO26, speed bump detection, pothole detection, small object detection, GhostConv, VoVGSCSPC, lightweight detector, in-video detection, object tracking, data leakage.

TABLE OF CONTENTS

TITLE **PAGE NO.**

ACKNOWLEDGMENT ..... **v**

ABSTRACT ..... **vi**

LIST OF FIGURES ..... **viii**

LIST OF TABLES ..... **viii**

LIST OF ABBREVIATIONS ..... **viii**

1. INTRODUCTION **1**

1.1 Background of the Project 1

1.2 Problem Statement 2

1.3 Motivation 2

1.4 Objectives 3

1.5 Scope of the Project 3

2. LITERATURE REVIEW **5**

2.1 Introduction 5

2.2 Review of Related Studies 5

2.3 Techniques and Technologies Used in Existing Systems 9

2.4 Research Gap 11

2.5 Summary 11

3. CHAPTER 1 – NEED FOR PROPOSED TECHNIQUE **12**

3.1 Existing Techniques 12

3.2 Limitations of Existing Techniques 13

3.3 Proposed Technique 13

3.4 Objectives of the Proposed Technique 14

3.5 Advantages of the Proposed Technique 15

4. CHAPTER 2 – DESIGN OF PROPOSED SYSTEM AND ITS IMPLEMENTATION **16**

4.1 Introduction 16

4.2 System Requirements 16

4.2.1 Hardware Requirements 16

4.2.2 Software Requirements 17

4.3 System Architecture 17

4.4 System Components/Modules 19

4.4.1 Data Acquisition and Augmentation Module 19

4.4.2 Lightweight Backbone and Neck Module 20

4.4.3 Small-Object Detection Head Module....20
4.4.4 Training and Distillation Module....20
4.4.5 Deployment and Inference Module....20
4.4.6 Baseline Benchmarking Module....20
4.5 Proposed Methodology....21
4.6 System Workflow....21
4.7 Algorithms/Techniques Used....22
4.8 Dataset/Data Collection....26
4.9 Data Preprocessing....28
4.10 Model/System Development....29
4.11 System Design Diagrams....31
4.12 Implementation....32
4.13 Coding/Program Implementation....33
4.14 User Interface / Application Screens....39
4.15 Chapter Summary....41
5. CHAPTER 3 – RESULTS AND DISCUSSION....42
5.1 Introduction....42
5.2 Experimental Setup....42
5.3 Evaluation Metrics....43
5.4 Experimental Results....43
5.4.1 Baseline Benchmark and Choice of Base Model....43
5.4.2 Ablation Study of GV-YOLO26....46
5.4.3 Effect of Knowledge Distillation....48
5.4.4 Inference Latency and Computational Efficiency....50
5.4.5 Qualitative Detection Results....50
5.5 Comparison with Existing Techniques....51
5.6 Discussion of Results....52
5.7 Chapter Summary....54
6. CHAPTER 4 – CONCLUSION AND FUTURE SCOPE....55
6.1 Summary of the Project....55
6.2 Conclusion....57
6.3 Future Scope....57
SDG GOALS....59
PUBLICATIONS....61
REFERENCES....62

LIST OF FIGURES

FIGURE NO. TITLE PAGE NO.

Fig. 4.1 Overall system architecture.....18

Fig. 4.2 Structure of the proposed GV-YOLO26 network.....19

Fig. 4.3 Project workflow.....22

Fig. 4.4 GhostConv and GSConv, the building blocks of the GV-YOLO26 backbone and neck 23

Fig. 4.5 A conventional decoupled head compared with the proposed LGDH.....25

Fig. 4.6 Use case diagram.....31

Fig. 4.7 Data flow diagram (level 1).....32

Fig. 4.8 GV-YOLO26 detections on held-out road images: (a) a distant bump, (b) a painted bump.....40

Fig. 4.9 GV-YOLO26 detections on held-out dashcam frames: (a) an urban road, (b) a distant pothole.....40

Fig. 4.10 Frames from the in-video detection demo with confirmed hazards and the warning banner.....41

Fig. 5.1 Overall accuracy of the baseline detectors on the public test split.....44

Fig. 5.2 Per-class AP@0.5 of the baseline detectors.....45

Fig. 5.3 Baseline detections on two test images: (a) a distant unpainted bump, (b) a pothole cluster.....46

LIST OF TABLES

TABLE NO. TITLE PAGE NO.

Table 2.1 Summary of representative prior work....8
Table 3.1 Comparison of existing techniques....13
Table 4.1 Hardware requirements....16
Table 4.2 Software requirements....17
Table 4.3 Feature pyramid levels for a 640 × 640 input....24
Table 4.4 SBP-YOLO dataset specification and split breakdown....27
Table 4.5 GV-YOLO26 design stages and model configurations....29
Table 4.6 GV-YOLO26 training configuration....30
Table 4.7 Baseline training configuration (identical across models)....30
Table 5.1 Baseline accuracy and cost on the SBP-YOLO test split....44
Table 5.2 Per-class accuracy of the baselines on the SBP-YOLO test split (%)....45
Table 5.3 Ablation study of GV-YOLO26 on the leakage-free held-out set....47
Table 5.4 Per-class accuracy of the design stages and accuracy on the raw test split (%)....48
Table 5.5 Effect of knowledge distillation....49
Table 5.6 In-video detection on the four test clips....51
Table 5.7 Comparison with existing techniques....52
Table 6.1 Achievement of project objectives....56

LIST OF ABBREVIATIONS

ABBREVIATION	EXPANSION
ADAS	Advanced Driver Assistance System
AMP	Automatic Mixed Precision
AP	Average Precision
BCKD	Bridging Cross-task Knowledge Distillation
CNN	Convolutional Neural Network
COCO	Common Objects in Context (dataset)
CWD	Channel-Wise Distillation
DFD	Data Flow Diagram
DFL	Distribution Focal Loss
FN / FP / TP	False Negative / False Positive / True Positive
FP16 / FP32	16-bit / 32-bit floating point
FPS	Frames Per Second
GFLOPs	Giga Floating-Point Operations
GPU	Graphics Processing Unit
GSConv	Group-Shuffle Convolution
GV-YOLO26	Ghost–VoVGSCSPC YOLO26 (the proposed model)
INT8	8-bit integer precision
IoU	Intersection over Union
ITS	Intelligent Transportation Systems
KD	Knowledge Distillation
LGDH	Lightweight Grouped Detection Head
mAP	mean Average Precision
MuSGD	Hybrid Muon and SGD optimiser
NMS	Non-Maximum Suppression
NWD	Normalized Gaussian Wasserstein Distance
P2–P5	Feature pyramid levels at strides 4 to 32
ProgLoss	Progressive Loss Balancing
RT-DETR	Real-Time Detection Transformer
SBP-YOLO	Speed Bump and Pothole YOLO
SDG	Sustainable Development Goal
STAL	Small-Target-Aware Label Assignment
VoVGSCSPC	VoV-style GSConv Cross-Stage Partial block

YOLO	You Only Look Once

1. INTRODUCTION

1.1 Background of the Project

Road surfaces carry two kinds of anomalies that matter to a moving vehicle. Some are defects, such as potholes and cracks, which appear as the pavement wears out. Others are built on purpose, such as speed bumps and humps, which are placed near schools, hospitals, junctions and residential streets to force traffic to slow down. Both share the same problem for a driver: if they are not seen in time, the vehicle hits them at the wrong speed.

On Indian roads, speed bumps are among the least standardised features a driver meets. A large share are unpainted, many do not follow the height and profile that the Indian Roads Congress specifies, and a single stretch of road can carry several of them with no signboard at all. At night, in rain, or behind a larger vehicle, an unmarked bump is practically invisible until the suspension finds it. For two-wheelers, which carry most of the road users in Tamil Nadu, meeting one at speed can mean anything from discomfort to a loss of control [1], [40].

Early detection systems used accelerometers and gyroscopes to sense the jolt as a vehicle crossed a bump [7]. Later systems used cameras, first with hand-crafted image processing [8] and then with deep learning [10], [43]. A camera can see a bump before the vehicle reaches it, and this matters for more than driver warnings. Modern vehicles with semi-active or active suspension can adjust their damping in advance of an uneven surface, an idea called suspension preview control [36]. For that to work, the vehicle needs to know where the next bump or pothole is, early and reliably.

This project proposes GV-YOLO26, a lightweight detector designed for exactly this setting. It starts from YOLO26n, the newest end-to-end model in the YOLO family, and redesigns its backbone, neck, head and training so that small, distant road anomalies become easier to detect while the model stays small enough to run at high frame rates on an embedded board. The choice of YOLO26n as the base is itself grounded in a

benchmark of seven stock detectors that we trained on a public dataset under a single protocol.

1.2 Problem Statement

A speed bump seen from a useful stopping distance is a thin, low-contrast band that may be only a dozen pixels tall in a 640-pixel frame. Potholes are similar: small, irregular and often clustered together. Standard detectors find most of their objects through feature maps that have been shrunk by a factor of 8 to 32, and at those scales a distant bump covers one feature cell or less. At the same time, a detector meant for a vehicle has to run on an embedded board in real time, which rules out simply making the model bigger.

The problem this project addresses is therefore how to detect speed bumps and potholes accurately at small scales while keeping the detector light enough for real-time embedded use, and how much a purpose-built lightweight design gains over the off-the-shelf detectors a developer would otherwise reach for.

1.3 Motivation

Unmarked bumps are a common sight on roads in and around Salem, as in most Indian towns, and most riders learn their positions only by repeatedly hitting them. A camera-based system would help in exactly the cases where a driver cannot see the hazard. Recent work has shown that a road anomaly detector can be aimed at a concrete use, suspension preview, and judged by its speed on real embedded hardware rather than by accuracy alone [24], [28]. We wanted to build such a detector ourselves on the newest available base model, and to see how far a carefully chosen set of lightweight and small-object techniques could take it.

A second motivation came from our own experiments with general-purpose detectors. When we trained several stock YOLO generations on a public speed bump and pothole dataset, the newer versions improved only slightly on the older ones. That raised a natural question: if moving from YOLOv8 to YOLO12 changes little, can a design built specifically for this task do better? The newest model in our benchmark, YOLO26n, was

also the most accurate and the cheapest, which made it the natural starting point for answering that question.

1.4 Objectives

The specific objectives of the project are:

1. To review existing speed bump and pothole detection methods, with attention to small-object detection and lightweight design.
2. To select a base detector on evidence by training stock real-time detectors on a public speed bump and pothole dataset under one fixed protocol.
3. To design GV-YOLO26 by rebuilding the YOLO26n backbone and neck with GhostConv and VoVGSCSPC blocks and adding a P2 detection level with a lightweight grouped detection head.
4. To train GV-YOLO26 with an NWD-augmented box loss, weather augmentation and knowledge distillation, and to measure the contribution of each change through an ablation study.
5. To evaluate the computational efficiency, parameter count, and inference latency of the proposed model in PyTorch FP32, and outline optimization pathways including TensorRT export for embedded automotive hardware.
6. To demonstrate in-video detection on dashcam-style video, linking detections across frames and warning only about hazards confirmed over several frames.

1.5 Scope of the Project

The project covers the detection of two classes, speed bumps and potholes, in road images and video from a monocular forward-facing camera. It includes the benchmarking of stock detectors on the SBP-YOLO dataset, the design, training and evaluation of GV-YOLO26, its computational profiling, an in-video detection demo with temporal confirmation, and an analysis of the results.

Some things are deliberately outside the scope. We did not build a physical in-vehicle device or connect the detector to a real suspension controller. Embedded hardware

deployment and TensorRT quantization tuning on Jetson platforms, along with monocular distance estimation, are outside the scope of this experimental evaluation and are detailed as future work in Chapter 6.

2. LITERATURE REVIEW

2.1 Introduction

This chapter reviews earlier work on detecting speed bumps and potholes, and on the techniques that GV-YOLO26 builds on. The aim is to understand which sensing approaches have been tried, which detection models have been used, how results have been reported, and what is still missing. The review is organised by sensing method first and detection model second, then covers the lightweight and small-object techniques that matter for this project, and ends with the research gap.

Two broad surveys give a useful starting map of the field. Saffarini et al. [34] grouped road anomaly detection into accelerometer-based and vision-based techniques and covered potholes, cracks and speed bumps together. Assemlali et al. [4] carried out a systematic review of 95 computer vision studies on road obstacles published between 2015 and 2024, and found that most relied on image input and that only about a third used public datasets. Both surveys note that comparisons across studies are difficult because datasets and evaluation settings differ.

2.2 Review of Related Studies

Sensor-based detection. The earliest practical systems did not use cameras at all. When a vehicle crosses a bump, its accelerometer and gyroscope record a clear vertical spike. Celaya-Padilla et al. [7] extracted accelerometric features from such signals and used a genetic algorithm to select the features that best separate bump events from normal driving. The method works in near real time and can be used to build maps of bump locations. Its limitation is structural: detection only happens after contact, so it helps the next vehicle rather than the current one. Darwiche and El-Hajj-Chehade [8] combined image processing with signal processing so that bumps could be detected without special sensors or GPS, an early move towards vision.

Stereo vision and distance estimation. A second camera lets a system estimate how far away a bump is. Varma et al. [43] used a ZED stereo camera with GPU-based deep learning to detect both marked and unmarked humps and estimate their distance in real

time. Peralta-López et al. [32] also used a ZED camera, designed a lightweight network for potholes, and built a database that separates potholes from speed bumps. Vasisht et al. [44] combined YOLOv8x with a ZED stereo camera and reported 93.1% mAP@50 at about 30 FPS on a laptop GPU. Distance is valuable for suspension control, but a stereo camera together with a discrete GPU is too costly for a simple retrofit device.

Monocular deep learning for speed bumps. Dewangan and Sahu [10] built a CNN-based speed bump system that runs on a Raspberry Pi and reported 98.54% accuracy. That figure is for image-level classification rather than box-level detection, so it cannot be compared with mAP. Magdy et al. [27] compared Faster R-CNN, VGG-16, MobileNet-SSD and two YOLOv3 variants for combined lane and bump detection, with YOLOv3 Lite-MobileNet best at 89.17%. Aishwarya et al. [1] focused on Indian roads. They compared Faster R-CNN with YOLOv5, treated marked and unmarked bumps separately and dealt with confusion between bumps and pedestrian crossings, at 18.5 ms per frame on a Jetson board.

Modified base detectors. A second group of studies modifies a base detector, which is also the approach taken in this project. Xiang et al. [50] built CRSCCG-YOLOv5s to detect several types of speed bump defect and reported 96.4% mAP with 8.8 million parameters. Wang et al. [48] added FasterNet and Dynamic Snake Convolution to a YOLOv5-based network for connected vehicles, giving up a little accuracy for a large gain in speed. Hussein et al. [15] collected road images in Egypt and deployed their detector on a Jetson Nano at 81% mAP. Li et al. [23] added SE and EMA attention to YOLOv8n and improved mAP@0.5 by 8.9 points in low light and rain. Mao et al. [28] proposed SBD-YOLO, based on YOLOv9 with an Aggregated-MBConv module that cuts computation by 82.8% for suspension preview control.

Lightweight small-object detectors. Liang et al. [24], [26] modified YOLOv11n for speed bumps and potholes by reducing its computation with cheap convolution blocks, adding a P2 detection branch for small objects with an efficient shared head to offset its cost, and training with the NWD loss, weather augmentation and knowledge distillation. On their 7,500-image dataset they reported 87.0% mAP@0.5 after distillation, against

81.2% for YOLOv11n, and 139.5 FPS on a Jetson AGX Xavier with TensorRT FP16 [25]. Studies of this kind show that small-object accuracy and a low computational budget can be pursued together, but they are built on YOLO generations that still rely on Non-Maximum Suppression and Distribution Focal Loss, both of which YOLO26 has since removed.

Studies on Indian roads. Only a few studies work with Indian road conditions directly. Dharneeshkar et al. [11] built a pothole dataset from Indian roads and trained YOLO models on it. Aishwarya et al. [1] reported that unpainted bumps and pedestrian crossings that look like painted bumps were the main sources of error. The *Towards Safer Roads* review [40] also uses an unmarked Indian speed bump as an example of a hazard that conventional mapping systems miss. These studies agree that unmarked bumps are the hardest and most important case.

Detection for suspension preview control. A growing part of this literature is aimed not at drivers but at the suspension itself. Shen et al. [36] describe the usual structure of a vision-based preview system: a camera module, an image processing module, a detection module, and a control module that adjusts damping before the wheel reaches the anomaly. They point out that the detection module has to finish within the time budget of the control loop, which is why frame rate on embedded hardware matters as much as accuracy. SBD-YOLO [28] and the detector of Liang et al. [24] were both designed for this module, on YOLOv9 and YOLOv11n respectively. Varma et al. [43] and Peralta-López et al. [32] approach the same application from the stereo side, adding distance to the detection.

Pothole detection and comparative studies. Pothole detection has been studied far more than speed bump detection. Park et al. [31] compared YOLOv4, YOLOv4-tiny and YOLOv5s and found the tiny model best once speed was taken into account. Mohan Prakash and Sriharipriya [29] tested YOLOX for potholes and found that the YOLOX-Nano model gave high accuracy at low computational cost. Wang et al. [49] improved a YOLOv3-based network with a ResNet backbone and tested robustness to adversarial inputs. Asad et al. [3] compared YOLOv1 to YOLOv5 and SSD-MobileNetV2 on OAK-

D and Raspberry Pi hardware, one of the few studies to treat edge hardware as part of the experiment.

Jakubec et al. [16] compared YOLO and R-CNN models under rain, night and sunset conditions. Tamagusko et al. [38] compared YOLOv3, YOLOv4 and YOLOv5 on a fixed 665-image dataset. Bhavana et al. [5] added Sobel edge preprocessing to YOLOv8 and reported 99.1% accuracy. Khan et al. [18] applied YOLOv8 to pothole detection for autonomous vehicles and studied the effect of data augmentation. Aquib et al. [2] compared YOLOv5 to YOLOv8, Kothari et al. [20] compared YOLO11 and YOLO12 at three scales on real road photographs taken in different weather, and Pham et al. [33] compared YOLOv7 to YOLOv10 for general road damage. None of these comparative studies includes speed bumps.

Reviews of the area. Safyari et al. [35] reviewed vision-based pothole detection and concluded that hybrid methods combining image processing with machine learning gave the highest accuracy in the studies they examined. The *Towards Safer Roads* review [40] is the only review we found that focuses on speed bumps specifically, covering vision-based, sensor-based and hybrid approaches published between 2017 and 2024. Shen et al. [36] reviewed road surface perception for suspension preview control and stressed the need for information about bump height and pothole depth, not only position. Table 2.1 summarises the main studies discussed in this section.

Table 2.1 Summary of representative prior work

Ref.	Method	Target	Reported outcome
[7]	Accelerometer features + genetic algorithm	Speed bumps	Post-contact detection
[43]	ZED stereo camera + GPU	Humps, distance	Real-time with ranging
[10]	CNN on Raspberry Pi	Speed bumps	98.54% classification accuracy
[27]	YOLOv3 Lite-MobileNet	Lanes and bumps	89.17%
[1]	Faster R-CNN vs YOLOv5	Indian speed bumps	18.5 ms per frame (Jetson)
[50]	CRSCCG-YOLOv5s	Bump defects	96.4% mAP, 8.8 M parameters
[48]	YOLOv5-FPNet	Speed bumps	Large FPS gain
[15]	YOLO on Jetson Nano	Speed bumps (Egypt)	81% mAP

Ref.	Method	Target	Reported outcome
[23]	YOLOv8n + SE/EMA	Low-light speed bumps	+8.9 mAP@0.5
[28]	SBD-YOLO (YOLOv9)	Speed bumps	82.8% less computation
[44]	YOLOv8x + stereo camera	Bumps and rumblers	93.1% mAP@50, 30 FPS
[24]	SBP-YOLO (YOLOv11n)	Bumps and potholes	87.0% mAP@0.5, 139.5 FPS
[31]	YOLOv4 / v4-tiny / v5s	Potholes	Tiny variant best
[3]	YOLOv1–v5, SSD	Potholes (edge)	OAK-D, Raspberry Pi
[16]	YOLO vs R-CNN	Potholes, weather	Condition-wise results
[20]	YOLO11 vs YOLO12	Potholes	Multi-weather comparison
[5]	POT-YOLO (YOLOv8 + Sobel)	Potholes	99.1% accuracy

Note: Metrics are as reported by the respective authors on their own datasets and hardware, and are not directly comparable across rows.

2.3 Techniques and Technologies Used in Existing Systems

One-stage detectors and the YOLO lineage. One-stage detectors such as SSD and YOLO predict boxes and classes in a single pass, which makes them the usual choice for real-time work on vehicles [1], [27]. YOLOv8 [17] introduced an anchor-free head and remains the most widely used baseline in road anomaly work. YOLOv9 [46] added programmable gradient information, YOLOv10 [45] trained a one-to-one head so that inference no longer needs Non-Maximum Suppression, and YOLO11 [19] refined the backbone and neck blocks. YOLO12 [39] replaced much of the convolutional backbone with area attention. YOLO26 [41], [42], the base of this project, removed Distribution Focal Loss, made inference end-to-end without Non-Maximum Suppression, and added progressive loss balancing (ProgLoss), small-target-aware label assignment (STAL) and the MuSGD optimiser. RT-DETR [53] brought transformer-based detection to real-time speeds.

Cheap convolutions. GhostNet [13] observed that many feature maps inside a CNN are near-copies of each other, and proposed generating part of them with cheap depthwise operations instead of full convolutions. The resulting GhostConv module produces the same number of output channels at roughly half the cost. GSConv, introduced in the slim-neck design of Li et al. [22], mixes a standard convolution with a depthwise convolution and then shuffles the channels, so that the cheap half still receives

information from the dense half. VoVNet [21] showed that one-shot aggregation of intermediate features is more efficient on GPUs than repeated dense connections, and VoVGSCSPC combines that idea with GSConv.

Small-object techniques. Because a distant bump occupies very few pixels, detectors for this task add features designed for small objects. Adding a detection level at stride 4 (P2) gives the head a feature map with four times the area of the usual smallest level. Li et al. [23] instead add attention to recover faint targets in poor lighting. On the loss side, Wang et al. [47] proposed the Normalized Gaussian Wasserstein Distance (NWD), which models each box as a two-dimensional Gaussian and measures the distance between them. Unlike IoU, NWD changes smoothly when a small predicted box is shifted by a few pixels, so it gives a more useful training signal for tiny objects.

Knowledge distillation and augmentation. Knowledge distillation trains a small student network to imitate a larger teacher [14]. For dense detection, Channel-Wise Distillation (CWD) [37] aligns the channel activations of intermediate feature maps, and BCKD [51] handles the mismatch between classification and localisation outputs when distilling detector heads. Albumentations [6] is a widely used library for image augmentation, including synthetic rain, snow, blur and lighting changes.

How results are reported. The way results are reported varies a great deal across the field. Some studies report image classification accuracy [5], [10], some report mAP at an IoU threshold of 0.5, and only a few report the stricter mAP@0.5:0.95. Speeds are reported on Raspberry Pi boards, Jetson modules and laptop GPUs, with and without TensorRT optimisation [3], [24], [44]. Datasets differ too: many speed bump studies use private collections, while others use public datasets hosted on Roboflow Universe [9]. Assemlali et al. [4] found that only about a third of the computer vision studies they reviewed used public datasets. This variety is the main reason results from different papers cannot be placed side by side, and it is why every model in this report is trained and evaluated under one protocol on one public dataset.

Hardware and deployment. Studies have deployed detectors on Raspberry Pi boards [3], [10], Jetson Nano and Jetson AGX Xavier modules [15], [24] and laptop GPUs [44].

Speed-ups usually come from NVIDIA TensorRT [30], which compiles a trained network into an optimised engine for a specific GPU and supports reduced-precision inference in FP16 and INT8.

2.4 Research Gap

Reading these studies together reveals four gaps that motivate this project.

1. Most speed bump and pothole detectors are general-purpose models applied to road images. Few are designed around the small size of distant targets, which is the case that matters most for early warning.
2. Detectors that add small-object features usually become heavier, and lightweight detectors usually lose small-object accuracy. Studies that address both together, at embedded speeds, are rare, and those that do are built on older YOLO generations that still depend on NMS and Distribution Focal Loss.
3. Embedded speed is often reported for one proposed model only, and frequently on laptop or desktop GPUs rather than the boards used in vehicles.
4. Results are reported on different, often private datasets, so it is hard to tell how much a proposed model gains over simply choosing a newer stock detector.

2.5 Summary

The literature shows a clear move from sensor-based detection to camera-based one-stage detectors, and from general road damage towards specific targets and specific uses such as suspension preview. It also shows that small-object accuracy and lightweight design pull in opposite directions, and that the few designs which reconcile them have not yet been carried over to the newest end-to-end detectors. Chapter 3 explains how the proposed GV-YOLO26 addresses this.

3. CHAPTER 1 – NEED FOR PROPOSED TECHNIQUE

3.1 Existing Techniques

This section describes the main existing techniques in terms of how they work, along with their advantages and disadvantages.

(a) Vibration and inertial sensing. An accelerometer and gyroscope, either built into the vehicle or inside a smartphone fixed to the dashboard, record vertical acceleration. A bump produces a characteristic spike that thresholds or classifiers can pick out [7]. The method is cheap and good for mapping, but the vehicle must already have hit the bump, and the signal changes with sensor mounting and suspension type.

(b) Classical image processing. Early camera systems used colour thresholds, edge detection and morphological operations to find the painted stripes of a bump [8]. These methods need no training data and run quickly, but they depend on paint and good lighting, and fail on unpainted bumps and in shadows.

(c) Stereo vision with deep learning. A stereo camera gives depth as well as appearance, so the system can estimate the distance to a bump [32], [43], [44]. The cost and calibration needs of a stereo rig, together with the GPU needed to run it, make it hard to fit into low-cost vehicles.

(d) General-purpose monocular detectors. Stock YOLO, SSD or Faster R-CNN models are trained on road images [1], [20], [31]. They are easy to train and deploy. However, their smallest detection level works on feature maps shrunk eight times, which loses distant bumps, and the larger variants that recover some of this accuracy are too slow for embedded boards.

(e) Modified lightweight detectors. Several studies add attention, lighter convolutions or extra branches to a YOLO model [23], [24], [28], [48], [50]. These run in real time on embedded boards, but most target either speed or small objects, not both, and those that target both are built on older YOLO generations with NMS-based post-processing. Table 3.1 summarises these techniques.

Table 3.1 Comparison of existing techniques

Technique	Advantages	Disadvantages
Vibration / inertial sensing	Low cost; no camera needed; good for mapping	Detects only after contact; depends on vehicle and mounting
Classical image processing	No training data; fast	Needs paint and good light; fails on unpainted bumps and shadows
Stereo vision + deep learning	Gives distance to the bump	Costly hardware; needs calibration and a GPU
General-purpose detectors	Easy to train and deploy	Weak on small, distant targets; larger variants too slow
Modified lightweight detectors	Real-time on embedded boards	Usually trade small-object accuracy for speed; built on older NMS-based bases

3.2 Limitations of Existing Techniques

The major limitations identified from the existing techniques can be stated together:

- Sensor-based methods react only after the vehicle crosses the bump, so they cannot warn the current driver or prepare the suspension.
- Hand-crafted vision methods depend on paint and lighting and do not generalise to unpainted bumps.
- Stereo systems are accurate but too costly for a simple retrofit device.
- General-purpose detectors lose small, distant targets at their coarse feature scales.
- Adding small-object features usually increases computation, which conflicts with embedded real-time use.
- Existing modified detectors are built on older YOLO generations that still depend on NMS, which adds latency and a threshold that has to be tuned at deployment.

3.3 Proposed Technique

The technique proposed in this project is GV-YOLO26, a lightweight real-time detector for speed bumps and potholes that we designed on top of YOLO26n. YOLO26n was chosen because it was both the most accurate and the cheapest of the seven stock detectors we benchmarked, and because its end-to-end head removes the NMS step from deployment.

GV-YOLO26 changes YOLO26n in four ways. First, the downsampling convolutions of the deeper backbone stages are replaced by GhostConv, and the C3k2 blocks at the P4 and P5 stages of the backbone and in the bottom-up path of the neck are replaced by VoVGSCSPC blocks built from GSConv, which lowers computation. Second, a fourth detection level, P2, is added at stride 4, giving the head a 160 × 160 feature map in which a distant bump covers several cells instead of a fraction of one. Third, every detection level uses a Lightweight Grouped Detection Head (LGDH) whose shared grouped convolutions offset most of the cost of the extra P2 level while keeping the one-to-one, NMS-free prediction of YOLO26. Fourth, training adds an NWD term to the box loss alongside small-target-aware label assignment, synthetic rain and snow augmentation for robustness, and knowledge distillation from a larger GV-YOLO26 teacher.

Around this, the project includes a measured baseline. Seven stock detectors, including YOLO26n itself, were trained on a public speed bump and pothole dataset under one fixed protocol, so that the gain of GV-YOLO26 can be judged against what a newer stock model alone achieves.

3.4 Objectives of the Proposed Technique

GV-YOLO26 addresses the limitations listed in Section 3.2 as follows:

- The P2 detection level, the NWD loss and small-target-aware label assignment target the small, distant bumps and potholes that general-purpose detectors lose.
- GhostConv, VoVGSCSPC and the LGDH reduce computation so that the extra P2 level does not push the model out of the embedded budget.
- Weather augmentation improves robustness to rain and snow, conditions in which camera-based detection usually degrades.
- Knowledge distillation lets the small model learn from a larger one without adding any cost at inference time.
- The NMS-free head of YOLO26, together with TensorRT export in FP16 or INT8 precision, makes real-time inference on a Jetson-class board practical and simplifies deployment.

- Every model, including the proposed one, is trained and evaluated under one protocol on a public dataset, so the gains can be checked and repeated.

3.5 Advantages of the Proposed Technique

- **Small-object accuracy:** the P2 level, the NWD loss and small-target-aware assignment all act on the case that general-purpose detectors handle worst, the small and distant target.
- **Low cost:** cheap convolutions and a grouped head pay for the extra P2 level: the deployed model has 1.57 million parameters and 5.1 GFLOPs, against 2.38 million and 5.3 GFLOPs for YOLO26n.
- **Simpler deployment:** one-to-one prediction removes NMS, so latency is predictable and no NMS threshold has to be tuned on the vehicle.
- **Early warning:** a single camera sees the bump before contact, unlike vibration-based methods, and without stereo hardware.
- **A strong starting point:** YOLO26n was the most accurate and the cheapest stock detector in our benchmark, so the improvements start from the best available base.

4. CHAPTER 2 – DESIGN OF PROPOSED SYSTEM AND ITS IMPLEMENTATION

4.1 Introduction

This chapter describes the design of the GV-YOLO26 system and its implementation. It covers the hardware and software, the overall architecture, the modules, the methodology and workflow, the algorithms inside GV-YOLO26, the dataset, preprocessing, model development, design diagrams and the code. Two pieces of work are described side by side: the proposed GV-YOLO26 model, and the benchmark of stock detectors that was used to choose its base and to measure its gains.

4.2 System Requirements

The requirements below cover the training environment used in this project and the deployment target.

4.2.1 Hardware Requirements

Table 4.1 Hardware requirements

Component	Specification	Purpose
GPU used in this project	2 × NVIDIA Tesla T4, 16 GB each (Kaggle)	Training (two runs in parallel), evaluation and latency profiling
Deployment target	NVIDIA Jetson AGX Xavier / Jetson Orin (planned)	Real-time TensorRT inference in the vehicle (future work)
Camera	Forward-facing vehicle or dashboard camera	Capturing road frames and video
Local machine	Laptop, Intel Core i5-1240P, 16 GB RAM	Code development, in-video demo on the CPU, analysis and reporting

4.2.2 Software Requirements

Table 4.2 Software requirements

Software	Version / detail	Purpose
Python	3.13.15 (Kaggle), 3.11 (laptop)	Programming language
PyTorch	2.11.0+cu128 (Kaggle)	Deep learning framework
GV-YOLO26 modules	Custom modules added to Ultralytics	GhostConv, VoVGSCSPC, LGDH, NWD loss and distillation
Ultralytics	8.4.156	YOLO26 base; training and evaluation of all models
CUDA	12.8	GPU acceleration
TensorRT	11.4 (export utility)	Model conversion for future embedded acceleration
Albumentations	2.0.8	Rain, snow, blur and lighting augmentation
OpenCV, NumPy, Matplotlib	OpenCV 4.14.0, NumPy 2.1.3	Video frames, result tables and plots
ByteTrack tracker	Built into Ultralytics	Linking detections across video frames
FFmpeg	System package	H.264 encoding of the annotated demo videos
Operating system	Linux	Execution environment

4.3 System Architecture

The overall architecture of the system is shown in Fig. 4.1. It has four layers. The data layer collects road images of speed bumps and potholes, applies weather and lighting augmentation, and resizes them to 640 × 640. The training layer builds the GV-YOLO26 network, trains it with the combined NWD and IoU box loss, and refines it with distillation from a larger teacher. The deployment layer takes the trained weights, exports them to a TensorRT engine in FP16 or INT8 precision, and loads that engine on the Jetson board in the vehicle.

The application layer is what runs while the vehicle moves. Each frame from the forward camera passes through the GV-YOLO26 engine, whose one-to-one head outputs final boxes directly without NMS, and the detections are passed to the suspension controller or shown as a warning. In this project the application layer was exercised by running the trained model on road images and video; a physical in-vehicle setup is future work.

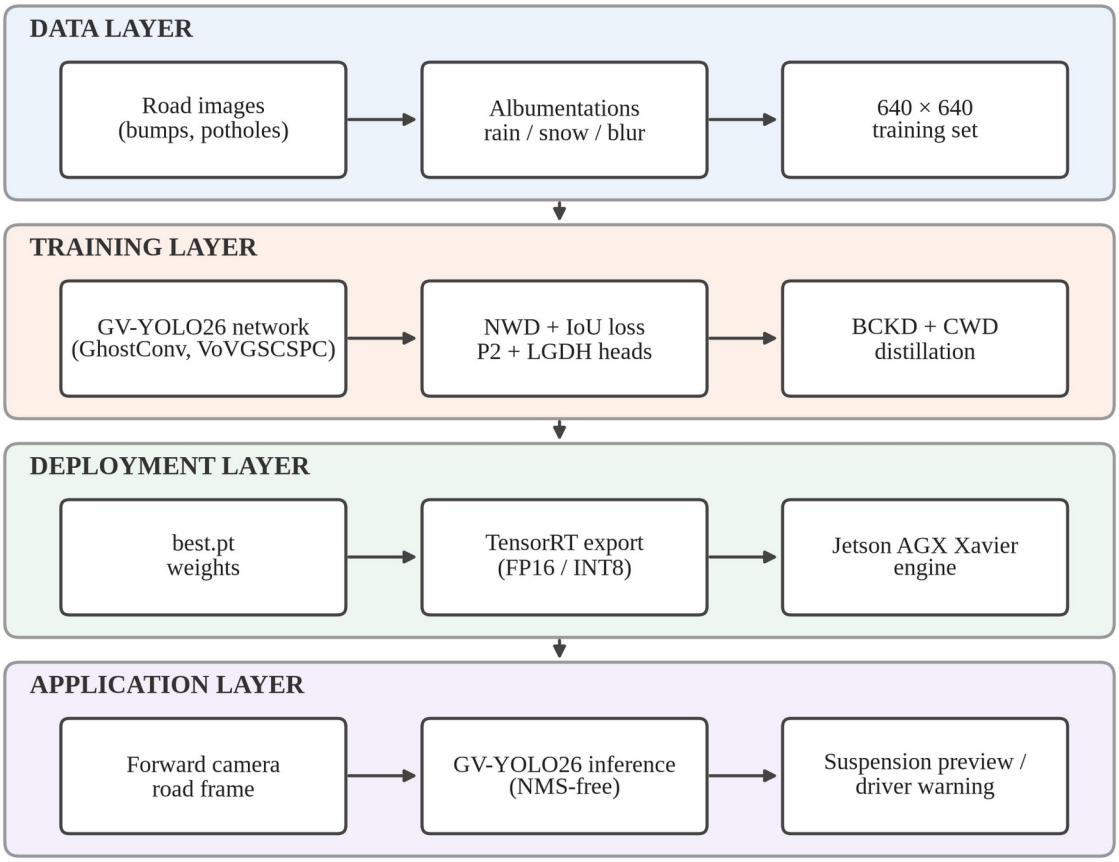


Fig. 4.1 Overall system architecture

The network itself is shown in Fig. 4.2. The backbone extracts features at four scales. Its early stages keep the standard convolutions and C3k2 blocks of YOLO26n, while its deeper downsampling layers are GhostConv and its P4 and P5 feature blocks are VoVGSCSPC; SPPF and C2PSA close the backbone. The neck first fuses these features top-down, extending the path down to the high-resolution P2 map so that it also receives semantic information from deeper levels, and then bottom-up again with GhostConv downsampling and VoVGSCSPC fusion at the deeper levels. The head applies an LGDH at each of the four levels, from the 160 × 160 P2 grid down to the 20 × 20 P5 grid, and its one-to-one branch outputs the final boxes without NMS.

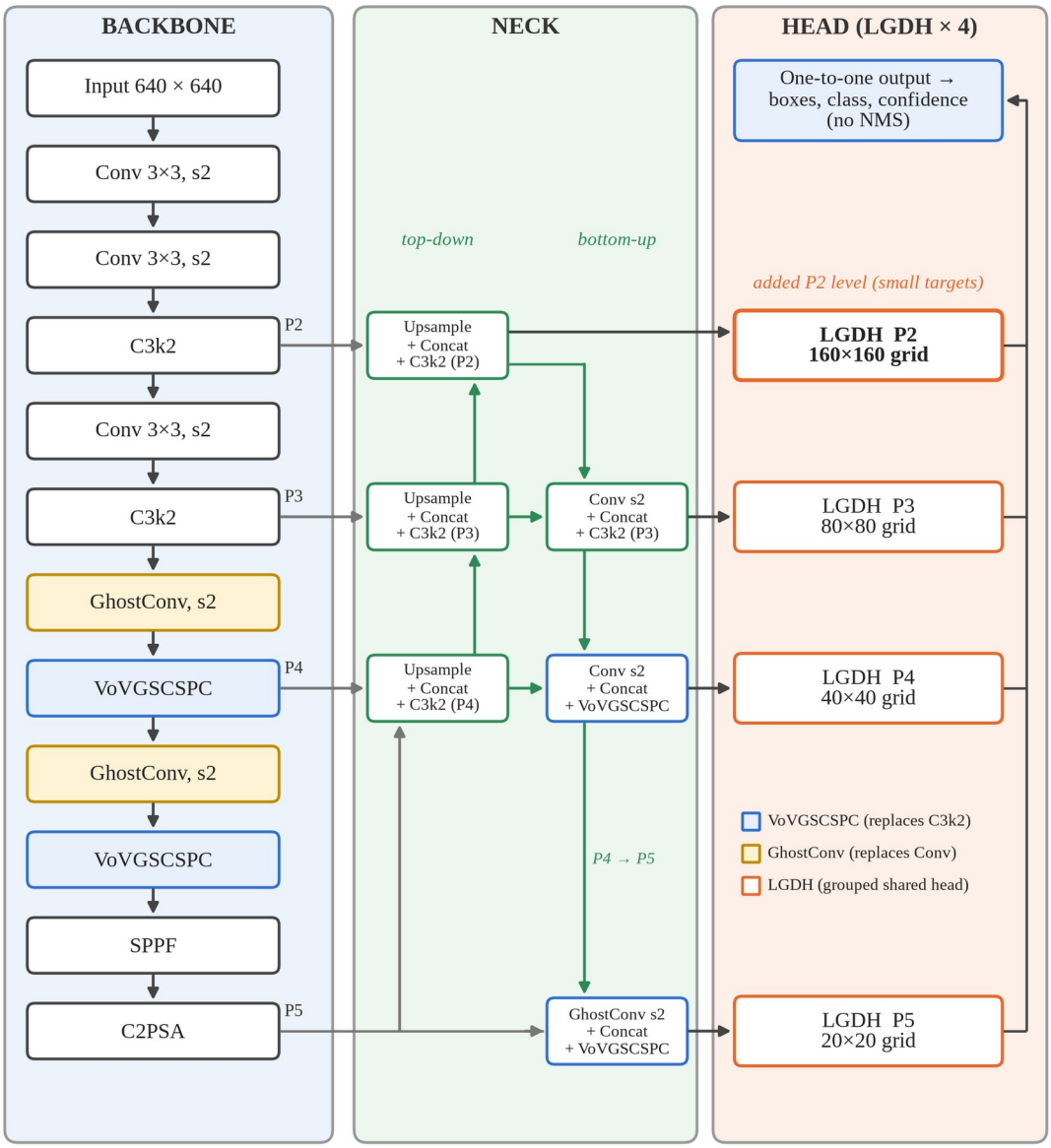


Fig. 4.2 Structure of the proposed GV-YOLO26 network

4.4 System Components/Modules

The system is divided into six modules.

4.4.1 Data Acquisition and Augmentation Module

This module gathers labelled road images and prepares them for training. All models in this project—including the stock baseline detectors and all GV-YOLO26 design stages—are trained and evaluated on the SBP-YOLO dataset of 7,600 images published on Kaggle as speed12 [12]. Images are normalized to 640 × 640 pixels. For GV-YOLO26, training is augmented with synthetic rain and snow, motion blur and photometric perturbations [6], in addition to standard mosaic, scaling, and flip augmentations.

4.4.2 Lightweight Backbone and Neck Module

This module extracts and fuses multi-scale features at low cost. GhostConv replaces the standard downsampling convolutions in the deeper stages of the YOLO26n backbone, and VoVGSCSPC replaces the C3k2 blocks at the P4 and P5 stages and in the bottom-up path of the neck.

4.4.3 Small-Object Detection Head Module

This module produces the boxes and class scores. It adds the P2 level at stride 4 to the usual P3, P4 and P5 levels, and uses an LGDH at each level. The LGDH uses two shared 3 × 3 grouped convolutions with eight channels per group, followed by two parallel 1 × 1 convolutions, one for box regression and one for classification, and keeps the one-to-one branch of YOLO26 so that inference needs no NMS.

4.4.4 Training and Distillation Module

This module trains the network. The box loss combines NWD with IoU so that small boxes receive a useful gradient, alongside the progressive loss balancing and small-target-aware label assignment of YOLO26. A teacher with the same architecture at the larger S scale was also evaluated through BCKD on the outputs and CWD on intermediate feature maps; Section 5.4.3 documents the behavior observed in practice.

4.4.5 Deployment and Inference Module

This module turns trained weights into a working detector. It loads the weights, runs inference on images or video frames, and draws the boxes with class names and confidence scores. For video, detections are linked across frames with the ByteTrack tracker [52], and a hazard is drawn, and a warning banner raised, only after its track has been seen in at least three frames (Section 4.7). For future vehicular deployment, export_trt.py provides conversion to TensorRT; because the head is end-to-end, the exported model produces final detections.

4.4.6 Baseline Benchmarking Module

This module trains and evaluates stock detectors under one fixed protocol on the same public dataset, so that the base model can be chosen on evidence and the improvements

of GV-YOLO26 can be judged against what an unmodified detector achieves. It records overall and per-class accuracy, model size, computation and latency for each model.

4.5 Proposed Methodology

The methodology followed in this project has these steps:

1. **Literature study:** review existing speed bump and pothole detection and the lightweight and small-object techniques available (Chapter 2).
2. **Baseline benchmark:** train seven stock detectors on a public speed bump and pothole dataset with one fixed configuration and evaluate them on the held-out test split.
3. **Base selection:** choose YOLO26n, the most accurate and cheapest baseline, as the starting point.
4. **Model design:** rebuild the backbone and neck with GhostConv and VoVGSCSPC, add the P2 level and the LGDH, and write one model configuration per design stage.
5. **Training:** train GV-YOLO26 stage by stage with the NWD-augmented loss and weather augmentation, and evaluate distillation from a larger teacher.
6. **Evaluation:** score each design stage once on a leakage-free held-out set, whose images share no source photo or video with the training data, and on the raw test split, overall and per class, and record parameter count, computation and PyTorch inference latency.
7. **Efficiency analysis and video demo:** profile PyTorch FP32 inference throughput on Tesla T4, demonstrate in-video detection with ByteTrack temporal confirmation, analyze failure modes, and formulate embedded TensorRT optimization as future work.

4.6 System Workflow

Fig. 4.3 shows the overall workflow of the project as a flowchart. The baseline benchmark selects the base model, after which GV-YOLO26 is designed, trained stage by stage, evaluated and deployed.

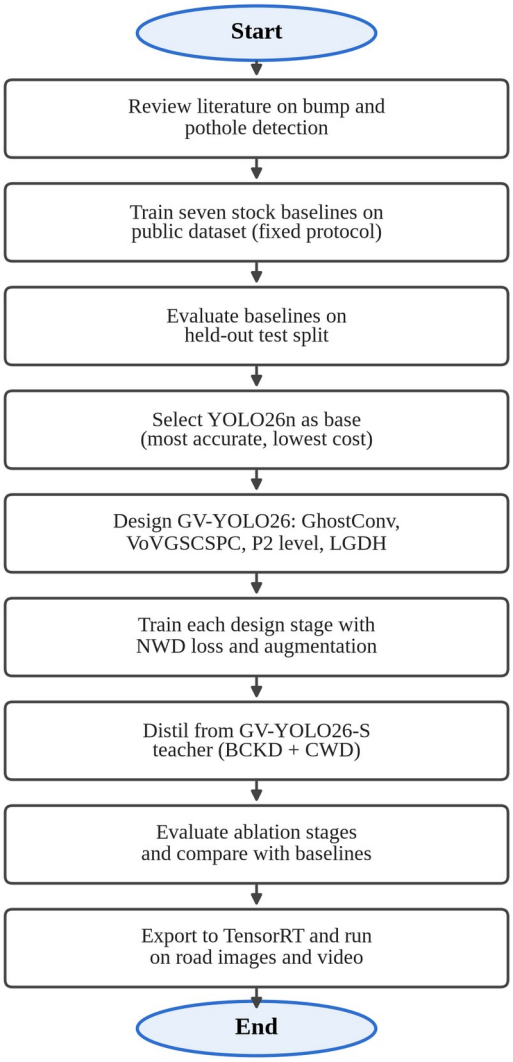


Fig. 4.3 Project workflow

4.7 Algorithms/Techniques Used

GhostConv. A standard convolution with C_2 output channels computes every output channel from all input channels. GhostConv computes only half of them this way, the intrinsic features, and produces the other half by applying a cheap 5 × 5 depthwise convolution to the intrinsic features [13]. The two halves are concatenated, as in Fig. 4.4(a). With a ratio of two, the cost of the layer is roughly halved:

$$Cost(GhostConv) \approx Cost(Conv) / s, \quad with\ s = 2 \tag{4.1}$$

GSConv and VoVGSCSPC. GSConv applies a standard convolution to produce half of the output channels, a depthwise convolution to produce the other half, concatenates them and shuffles the channels [22], as in Fig. 4.4(b). The shuffle mixes the dense and

cheap features so that information is not trapped in one half. VoVGSCSPC is a cross-stage partial block built from GSConv bottlenecks with one-shot aggregation in the style of VoVNet [21], [22]. In GV-YOLO26 it replaces the C3k2 block of YOLO26n at the P4 and P5 stages.

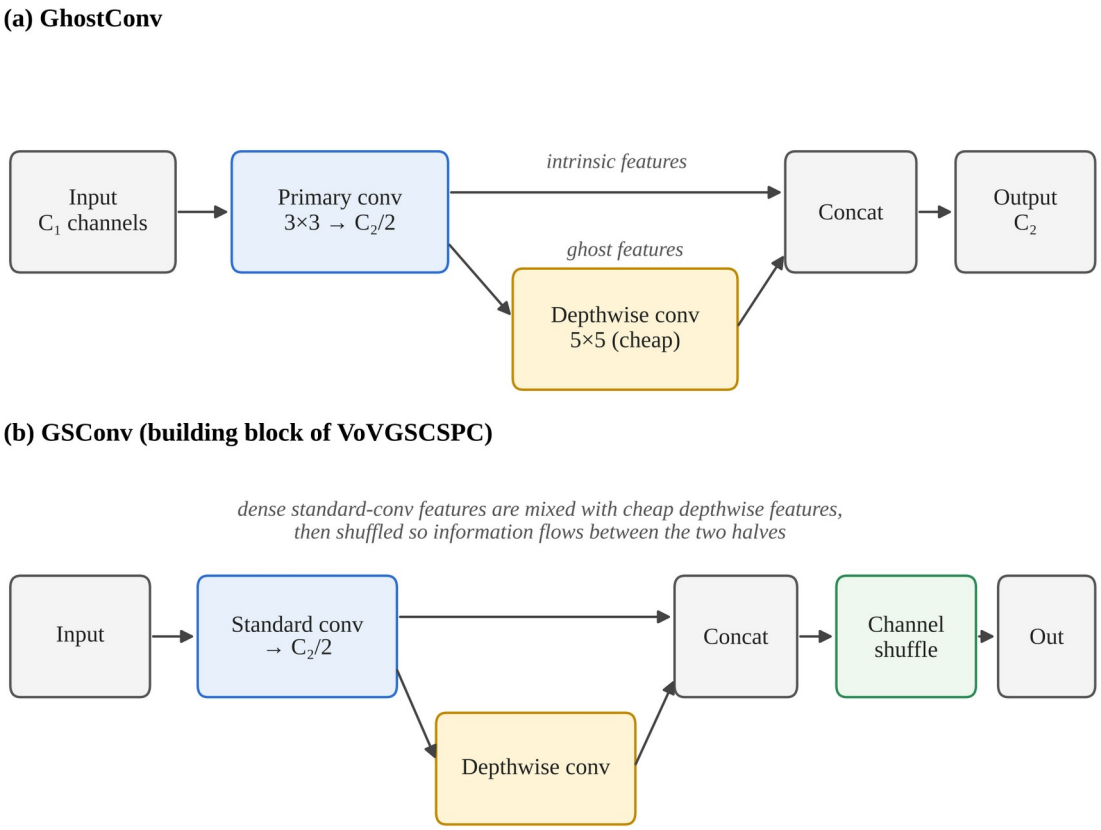


Fig. 4.4 GhostConv and GSConv, the building blocks of the GV-YOLO26 backbone and neck

The P2 detection level. Each detection level sees the image through a grid whose cells are as wide as the level's stride. Table 4.3 shows what this means for a bump 16 pixels tall in a 640-pixel frame. At P3, the smallest level of the standard YOLO26 head, the bump covers two cells; at P5 it covers half a cell. The P2 level gives it four cells, which leaves enough spatial detail for the head to locate it.

Table 4.3 Feature pyramid levels for a 640 × 640 input

Level	Stride	Grid size	Cells covering a 16-pixel-tall bump	In YOLO26n
P2	4	160 × 160	4	No (added in GV-YOLO26)
P3	8	80 × 80	2	Yes
P4	16	40 × 40	1	Yes
P5	32	20 × 20	0.5	Yes

LGDH. A conventional decoupled head runs two separate stacks of convolutions at each level, one for the box and one for the class. Adding a 160 × 160 level makes this expensive, because the cost of the head grows with the area of the feature map and the P2 map has four times the area of P3; in our measurements, adding the P2 level to YOLO26n raised the cost of the deployed model from 5.3 to 6.6 GFLOPs. The LGDH, shown in Fig. 4.5, shares two 3 × 3 grouped convolutions between the two outputs, an approach that has proved effective in recent lightweight road detectors [24]. Because the default YOLO26 head is already light, with a depthwise classification branch and only four box channels per location, the groups were made narrow: each filter sees eight channels (c/8 groups). With sixteen channels per group the LGDH was no cheaper than the default head, while with eight it brought the cost back to 6.2 GFLOPs. Two 1 × 1 convolutions then produce the box and class outputs. The same structure is used for the one-to-many branch used in training and the one-to-one branch used at inference.

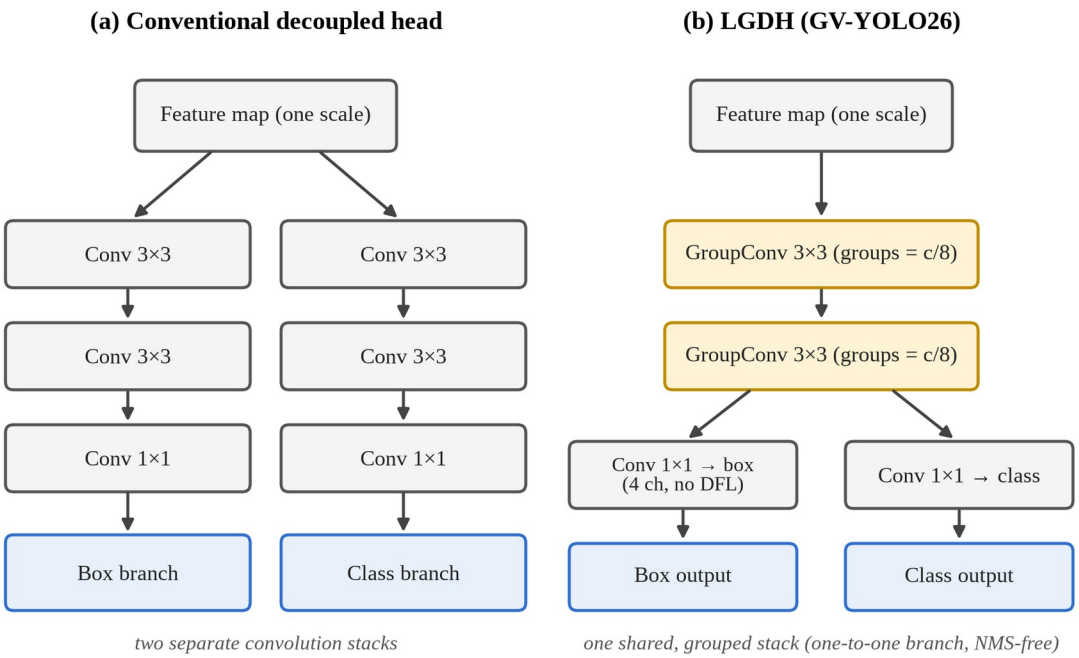


Fig. 4.5 A conventional decoupled head compared with the proposed LGDH

Normalized Gaussian Wasserstein Distance. IoU is very sensitive for small boxes: shifting a 10-pixel box by 3 pixels cuts its IoU sharply, and two small boxes that do not overlap have an IoU of zero no matter how close they are. NWD avoids this by modelling each box (cx, cy, w, h) as a two-dimensional Gaussian and measuring the second-order Wasserstein distance between the two Gaussians [47]:

$$W_2^2(N_a, N_b) = \| [cx_a, cy_a, w_a/2, h_a/2] - [cx_b, cy_b, w_b/2, h_b/2] \|^2 \quad (4.2)$$

$$NWD(N_a, N_b) = exp(-\sqrt{(W_2^2(N_a, N_b))} / C) \quad (4.3)$$

Here C is a normalising constant related to typical object size. GV-YOLO26 combines NWD with IoU in the box loss, with a weighting factor α between 0 and 1, set to 0.5 in this project:

$$L_{box} = (1 - \alpha) \cdot (1 - NWD) + \alpha \cdot L_{IoU} \quad (4.4)$$

Knowledge distillation. The student (GV-YOLO26 at N scale) learns from a teacher of the same architecture at the larger S scale. Two terms are added to the normal detection loss: a BCKD term that aligns the student's classification and localisation outputs with the teacher's [51], and a CWD term that aligns the channel activations of intermediate feature maps [37]. Both are weighted by 0.5:

$$L = L_{det} + 0.5 \cdot L_{BCKD} + 0.5 \cdot L_{CWD} \quad (4.5)$$

Distillation adds nothing at inference time, because the teacher is only used during training.

Intersection over Union and NMS. A prediction is counted as correct when its IoU with a ground-truth box of the same class is above a threshold such as 0.5. Non-Maximum Suppression removes duplicate predictions by keeping the highest-scoring box of a class and discarding any other box whose IoU with it exceeds a threshold. The baselines from YOLOv8n to YOLO12n rely on NMS; YOLO26n and GV-YOLO26 are trained with a one-to-one assignment so that each object receives a single prediction and NMS is not needed at inference.

$$IoU = Area(A \cap B) / Area(A \cup B) \tag{4.6}$$

TensorRT quantisation. TensorRT fuses layers, selects GPU kernels for the target board, and can store weights and activations in FP16 or INT8 instead of FP32 [30]. Lower precision cuts memory traffic and lets the Jetson's tensor cores run more operations per cycle, at a small risk to accuracy.

Temporal confirmation for video. A per-frame detector fires on single frames at shadows, patches and road markings. In the video demo, the boxes of each frame are associated with existing tracks by ByteTrack [52], which matches high-confidence boxes first and then uses low-confidence boxes to keep existing tracks alive, so a distant hazard is not lost when its score dips for a frame. A track t is shown only once it has been observed in at least k frames:

$$show(t) \Leftrightarrow frames(t) \geq k \text{ and } conf(t) \geq \tau, \quad k = 3, \tau = 0.35 \tag{4.7}$$

A one-frame false alarm therefore never reaches the driver, while a real hazard, which stays in view for many frames, is confirmed within about a tenth of a second at the 24 to 30 frames per second of a dashcam.

4.8 Dataset/Data Collection

The dataset used in this project is the SBP-YOLO Speed Bump and Pothole Detection Dataset [12], released on Kaggle by elangovan001 as speed12 (originally introduced by Liang et al. [23]). It contains 7,600 images with bounding-box annotations in YOLO

format across two classes: speed bumps (class 1) and potholes (class 0). The dataset is organized into train/ (5,725 images), valid/ (750 images), and test/ (1,125 images) directories. Table 4.4 details the dataset specifications and split partitions used across all experiments.

Table 4.4 SBP-YOLO dataset specification and split breakdown

Dataset	SBP-YOLO road hazard dataset (speed12) [12]
Target classes	Speed bump (class 1), Pothole (class 0)
Total images	7,600 images (12,205 potholes, 3,217 speed bumps)
Original directory structure	train/ (5,725) / valid/ (750) / test/ (1,125)
Training partition (train_used)	5,243 images (8,300 potholes, 2,144 speed bumps)
Validation split (val_carved)	586 images (10% seeded sample of train/ for checkpoint selection)
Raw test partition	1,125 images (1,802 potholes, 497 speed bumps)
Raw validation partition	750 images (1,197 potholes, 343 speed bumps)
Leakage-free held-out set	482 images (354 potholes, 384 speed bumps; zero shared photos/drives with train/)
Input resolution	640 × 640 pixels (RGB)
Dataset source	SBP-YOLO benchmark collection (Kaggle speed12) [12]
Role in project	Used for all stock baselines and all GV-YOLO26 design stages

Note: The validation split (586 images, seed 42) was partitioned from train/ for checkpoint selection, leaving 5,243 training images. The 482-image leakage-free held-out set was constructed from test/ and valid/ by filtering out duplicate augmentations and video frames sharing scenes with train/.

The SBP-YOLO dataset (speed12) comprises 7,600 images across train/, valid/, and test/ folders. Before evaluation, the dataset required rigorous verification. File names and image hashes show that many images in test/ and valid/ are augmented copies of photos in train/: for example, train/ holds add_gasuss_noise_1386_jpg.rf.df5cd630….jpg, a noise-augmented copy of the test image 1386_jpg.rf.df5cd630….jpg with an identical visual hash. Other files represent neighbouring frames from sequential dashcam or GoPro video drives. We traced every image back to its source photo and drive. Of the 1,125 test images, 526 share a source photo with train/ and 839 share a photo or video sequence; of the 750 valid images, 554 do. Scores on the raw test split therefore reward memorisation as well as detection. To ensure strict academic integrity, all main

experimental results in Chapter 5 are reported on the remaining 482 images (286 from test/ and 196 from valid/), which share zero source photos, videos, or drives with the training partition. This leakage-free held-out set contains 354 pothole instances in 145 images and 384 speed bump instances in 337 images. Each pothole in a cluster is annotated individually, and speed bump bounding boxes cover the full road width, conventions that make strict-IoU metrics lower for potholes than for speed bumps.

4.9 Data Preprocessing

Preprocessing follows the Ultralytics pipeline, with additional augmentation for GV-YOLO26:

- **Resizing:** each image is letterboxed to 640 × 640 and pixel values are scaled to the range 0 to 1.
- **Weather augmentation (GV-YOLO26):** synthetic rain and snow are added with Albumentations transforms so that the model sees adverse weather during training [6].
- **Photometric augmentation (GV-YOLO26):** motion blur and random changes in brightness, contrast and saturation, applied with a low probability on top of the standard recipe.
- **Standard augmentation (all models):** the default Ultralytics recipe, with HSV jitter, scaling, translation, horizontal flip and mosaic, mosaic switched off for the final ten epochs and vertical flip off. It was applied identically to every model and not tuned.
- **Label format:** YOLO format, with one line per object giving the class index and the normalised box centre, width and height.
- **Splits and evaluation:** a 10% validation split (586 images, seed 42) is drawn from the train/ folder for checkpoint selection, leaving 5,243 training images; the 482-image leakage-free held-out set is used for primary evaluation to prevent data leakage.

4.10 Model/System Development

GV-YOLO26 was built in steps from YOLO26n, with one model configuration file for each step, so that the contribution of each change can be measured in the ablation study of Chapter 5. Table 4.5 lists the stages.

Table 4.5 GV-YOLO26 design stages and model configurations

Stage	Change	Configuration file
1	YOLO26n baseline	yolo26n.yaml
2	Add P2 detection level	ablation/gv-yolo26n-p2.yaml
3	Replace head with LGDH	ablation/gv-yolo26n-p2-lgdh.yaml
4	Add GhostConv	ablation/gv-yolo26n-p2-lgdh-ghost.yaml
5	Add VoVGSCSPC (full architecture)	gv-yolo26n.yaml
6	Train with NWD loss	gv-yolo26n.yaml + --nwd

Note: Files are in the models/ folder; the letter after "yolo26" selects the scale, so gv-yolo26n.yaml and gv-yolo26s.yaml both read gv-yolo26.yaml. Stage 6 uses the architecture of stage 5 with the NWD term enabled in the box loss. The distilled model is trained from stage 6 with distill.py.

GV-YOLO26 was trained with the settings in Table 4.6. Each stage is trained for 150 epochs on the SBP-YOLO dataset, longer than the baseline schedule, because the new layers start from random weights; YOLO26n is retrained under the same schedule and on the same data so that the ablation compares like with like. Layers whose shapes match YOLO26n are initialised from its COCO-pretrained weights. Two stages were trained at a time, one on each of the two T4 GPUs of a Kaggle session, and the distillation runs used the S-scale teacher.

Table 4.6 GV-YOLO26 training configuration

Parameter	Value
Input resolution	640 × 640, letterboxed
Epochs	150, early stopping patience 30
Batch size	16
Optimiser	Auto-selected (same rule as the baselines)
Box loss	IoU combined with NWD, $\alpha = 0.5$ (equation 4.4)
Distillation run	BCKD 0.5 + CWD 0.5 (equation 4.5): from scratch for 150 epochs, and as a 50-epoch fine-tune of stage 6 (AdamW, learning rate 5×10^{-4})
Teacher	GV-YOLO26 at S scale (6.11 M parameters, 19.2 GFLOPs)
Hardware	2 × NVIDIA Tesla T4, 16 GB (Kaggle); one design stage per GPU
Framework	Ultralytics 8.4.156, PyTorch 2.11, Python 3.13.15
Data split	5,243 train / 586 val / 482 clean held-out (SBP-YOLO dataset)

Note: Random seed 42 with deterministic mode and AMP enabled, as for the baselines (Table 4.7).

For the baseline benchmark we trained seven stock detectors at their smallest scale: YOLOv8n, YOLOv9t, YOLOv10n, YOLO11n, YOLO12n, YOLO26n and RT-DETR-L [17], [19], [39], [41], [45], [46], [53]. All used the same configuration, shown in Table 4.7, so that only the architecture changed between runs.

Table 4.7 Baseline training configuration (identical across models)

Parameter	Value
Input resolution	640 × 640, letterboxed
Epochs	50, early stopping patience 15
Batch size	16 (8 for RT-DETR-L)
Optimiser	AdamW (auto-selected), learning rate 1.67e-3, momentum 0.9
Weight initialisation	COCO-pretrained, per architecture
Mixed precision	AMP enabled
Framework	Ultralytics 8.4.156, PyTorch 2.10, Python 3.12
Hardware	NVIDIA Tesla T4, 16 GB (Kaggle)
Random seed	42, deterministic

4.11 System Design Diagrams

The use case diagram in Fig. 4.6 shows the two actors. The researcher or developer prepares data, trains GV-YOLO26, evaluates it and exports it for the vehicle. The driver or vehicle system uses the deployed detector and receives warnings or a suspension preview signal.

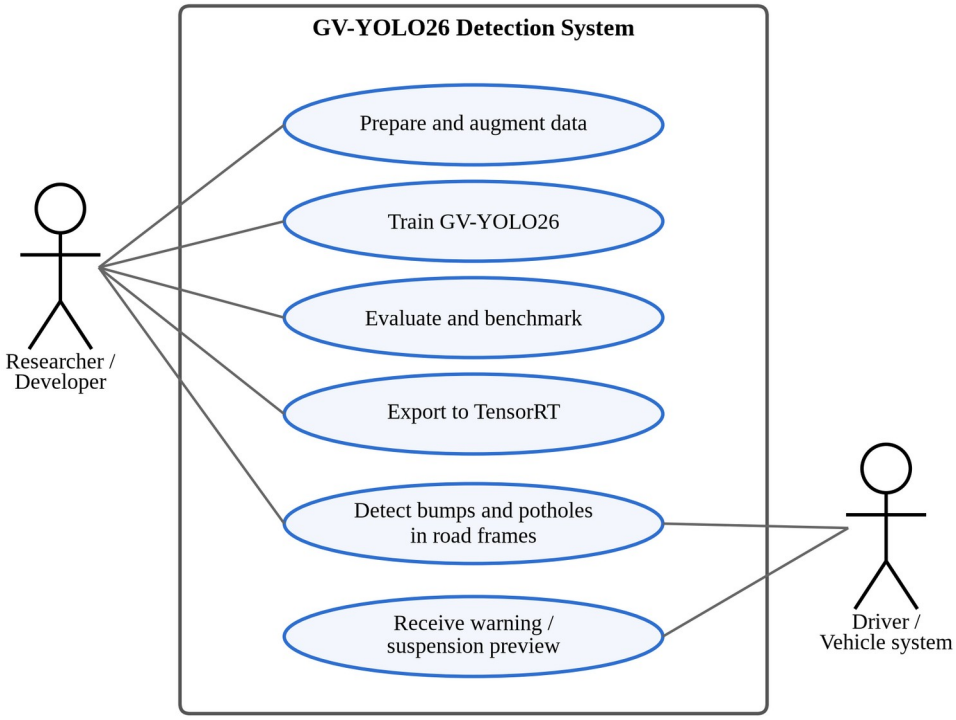


Fig. 4.6 Use case diagram

The data flow diagram in Fig. 4.7 shows how data moves through the system. Images and labels are prepared and stored (D1), training produces weights (D2), evaluation writes metrics and logs (D3), and the weights are exported to a TensorRT engine. At run time, camera frames flow into the real-time inference process, which sends alerts to the vehicle or driver.

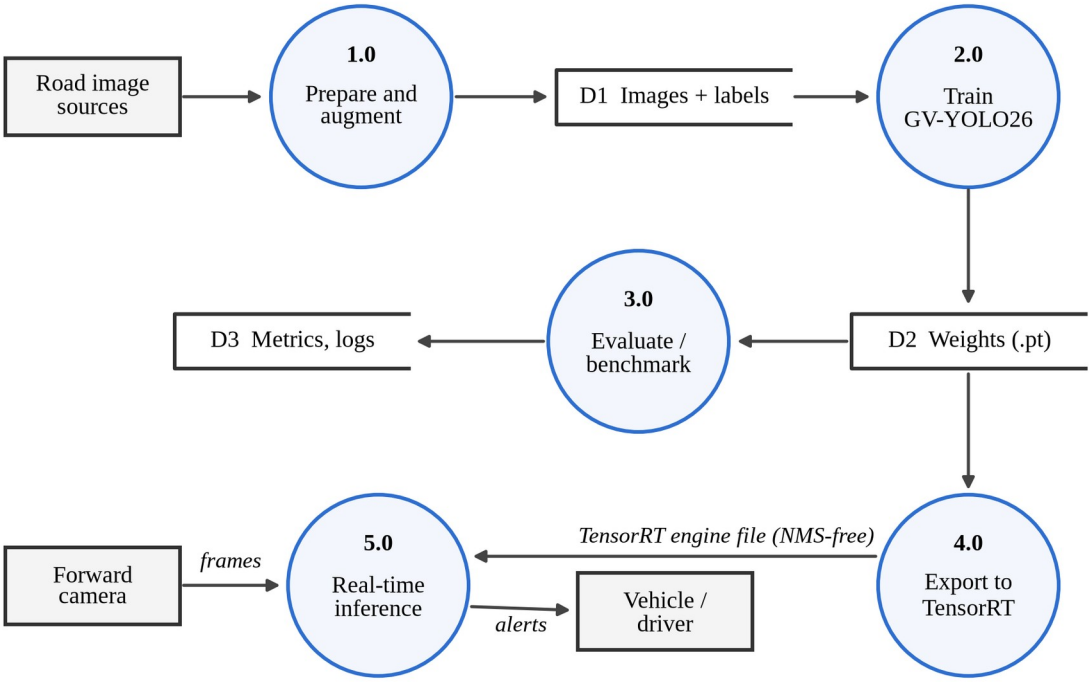


Fig. 4.7 Data flow diagram (level 1)

4.12 Implementation

GV-YOLO26 was implemented on the Ultralytics framework (version 8.4.156), which provides the YOLO26 base model. We wrote the new modules, GSConv, VoVGSCSPC and the LGDH head, in a separate Python package and registered them, together with GhostConv, with the Ultralytics model parser so that they can be used from a YAML model configuration. The NWD term was added to the box loss through a custom trainer with a weighting factor α, and the distillation stage was implemented as a second custom trainer that adds the BCKD and CWD losses using a frozen teacher.

Each design stage in Table 4.5 has its own configuration file, so building a stage only means changing the file passed to the trainer. Before training, every configuration was checked with tools/model_summary.py, which builds the model, runs a forward pass on a 640 × 640 input in training and inference mode to confirm that all four detection levels produce outputs of the expected size, and reports the parameter count and GFLOPs of both the training model and the fused deployment model. The full pipeline (training with the NWD loss, distillation, validation, inference and NMS-free ONNX export) was also run end to end on a small synthetic dataset before the real experiments.

All training was run on Kaggle notebooks with two Tesla T4 GPUs. After training, every checkpoint was measured by one script, tools/benchmark.py, under one standardized protocol: accuracy with the Ultralytics validator (confidence threshold 0.001, batch size 1) on the validation split, the raw test split and the leakage-free held-out set; the parameter count and GFLOPs of the deployed (fused) model; and the end-to-end time of 100 single-image predictions in PyTorch FP32 after 10 warm-up images. YOLO26n and GV-YOLO26 were evaluated and timed with their one-to-one head (nms=False), which is the NMS-free form they operate in. The leakage-free set was built by tools/make_clean_split.py, which removes every test and valid image whose source photo, video or drive also appears in the training folder.

4.13 Coding/Program Implementation

The project code is organised as shown below. The model configurations, custom modules and scripts are kept separate from the Ultralytics package (version 8.4.156), which is installed as a dependency; importing gv_modules registers the new layers with it.

```
GV-YOLO26/
├── models/
│   ├── gv-yolo26.yaml            GV-YOLO26 (N and S scales)
│   └── ablation/                 One file per design stage
├── gv_modules/                   GSConv, VoVGSCSPC, LGDH head, NWD and
│                                 distillation losses, model and trainers
├── tools/model_summary.py        Parameters and GFLOPs per stage
├── tools/make_clean_split.py     Leakage-free held-out set, dataset stats
├── tools/benchmark.py            Accuracy, cost and PyTorch latency
├── tools/make_figures.py         Report figures
├── train.py, distill.py          Training and distillation
├── val.py, infer.py              Evaluation, latency and inference
├── demo_video.py                 In-video detection with tracking
├── export_trt.py                 TensorRT export utility (future deployment)
├── Kaggle_Train_GV_YOLO26.ipynb  End-to-end Kaggle training notebook
├── benchmark/train_baselines.py  Seven stock baselines
└── data/data.yaml                Dataset configuration
```

GhostConv module. The first convolution produces half of the output channels; the second is a depthwise 5×5 convolution (groups equal to channels) that generates the cheap ghost features.

```
class GhostConv(nn.Module):
    def __init__(self, c1, c2, k=1, s=1, g=1, act=True):
        super().__init__()
        c_ = c2 // 2                                    # intrinsic channels
        self.cv1 = Conv(c1, c_, k, s, None, g, act=act)
        self.cv2 = Conv(c_, c_, 5, 1, None, c_, act=act)  # depthwise


    def forward(self, x):
        y = self.cv1(x)
        return torch.cat((y, self.cv2(y)), 1)
```

GSConv and VoVGSCSPC. GSConv concatenates a dense half and a depthwise half and then interleaves their channels. VoVGSCSPC splits its input into two 1×1 branches, passes one through a GSConv bottleneck with a depthwise shortcut (GSBottleneckC), and fuses both with a final 1×1 convolution.

```
class GSConv(nn.Module):
    def __init__(self, c1, c2, k=1, s=1, act=True):
        super().__init__()
        c_ = c2 // 2
        self.cv1 = Conv(c1, c_, k, s, act=act)              # dense half
        self.cv2 = Conv(c_, c_, 5, 1, None, c_, act=act)    # depthwise half


    def forward(self, x):
        x1 = self.cv1(x)
        x2 = torch.cat((x1, self.cv2(x1)), 1)
        b, n, h, w = x2.shape                               # channel shuffle
        return x2.view(b, 2, n // 2, h, w).transpose(1, 2).reshape(b, n, h, w)


class VoVGSCSPC(nn.Module):
    def __init__(self, c1, c2, n=1, e=0.5):
        super().__init__()
        c_ = int(c2 * e)
        self.cv1 = Conv(c1, c_, 1, 1)
        self.cv2 = Conv(c1, c_, 1, 1)
        self.gsb = nn.Sequential(*(GSBottleneckC(c_, c_) for _ in range(n)))
        self.cv3 = Conv(2 * c_, c2, 1)


    def forward(self, x):
        return self.cv3(torch.cat((self.cv2(x), self.gsb(self.cv1(x))), 1))
```

LGDH head. The head subclasses the YOLO26 Detect module, so it inherits the one-to-one branch, the decoding and the NMS-free post-processing. It replaces the default box and class stacks with a shared grouped stem per level and two 1 × 1 output convolutions, for both the one-to-many and the one-to-one branch.

```
class LGDH(Detect):
    channels_per_group = 8                    # groups = c / 8 in the stem

    def __init__(self, nc=80, reg_max=16, end2end=False, ch=()):
        super().__init__(nc, reg_max, end2end, ch)
        g = lambda c: max(c // self.channels_per_group, 1)
        self.stem = nn.ModuleList(
            nn.Sequential(Conv(x, x, 3, g=g(x)), Conv(x, x, 3, g=g(x)))
            for x in ch)
        self.cv2 = nn.ModuleList(
            nn.Sequential(nn.Conv2d(x, 4 * self.reg_max, 1)) for x in ch)
        self.cv3 = nn.ModuleList(
            nn.Sequential(nn.Conv2d(x, self.nc, 1)) for x in ch)
        if end2end:                           # copy for NMS-free inference
            self.one2one_stem = copy.deepcopy(self.stem)
            self.one2one_cv2 = copy.deepcopy(self.cv2)
            self.one2one_cv3 = copy.deepcopy(self.cv3)

    def forward_head(self, x, box_head=None, cls_head=None, stem=None):
        if box_head is None or cls_head is None:  # removed by fuse()
            return {}
        bs = x[0].shape[0]
        f = [stem[i](x[i]) for i in range(self.nl)]  # shared stem
        boxes = torch.cat([box_head[i](f[i]).view(bs, 4 * self.reg_max, -1)
                           for i in range(self.nl)], dim=-1)
        scores = torch.cat([cls_head[i](f[i]).view(bs, self.nc, -1)
                            for i in range(self.nl)], dim=-1)
        return {"boxes": boxes, "scores": scores, "feats": x}
```

Model configuration. The full GV-YOLO26 architecture is defined in one YAML file, models/gv-yolo26.yaml. Layer indices in the comments match the connections in Fig. 4.2; the last line attaches an LGDH to the P2, P3, P4 and P5 outputs. The S row of scales defines the distillation teacher.

```
nc: 2                                   # speed bump, pothole
end2end: True                           # one-to-one head, NMS-free inference
reg_max: 1                              # no DFL
scales:                                 # [depth, width, max_channels]
  n: [0.50, 0.25, 1024]
  s: [0.50, 0.50, 1024]
backbone:
  - [-1, 1, Conv, [64, 3, 2]]                 # 0-P1/2
  - [-1, 1, Conv, [128, 3, 2]]                # 1-P2/4
  - [-1, 2, C3k2, [256, False, 0.25]]         # 2
  - [-1, 1, Conv, [256, 3, 2]]                # 3-P3/8
  - [-1, 2, C3k2, [512, False, 0.25]]         # 4
  - [-1, 1, GhostConv, [512, 3, 2]]           # 5-P4/16
  - [-1, 2, VoVGSCSPC, [512]]                 # 6
  - [-1, 1, GhostConv, [1024, 3, 2]]          # 7-P5/32
  - [-1, 2, VoVGSCSPC, [1024]]                # 8
  - [-1, 1, SPPF, [1024, 5, 3, True]]         # 9
  - [-1, 2, C2PSA, [1024]]                    # 10
head:
  - [-1, 1, nn.Upsample, [None, 2, "nearest"]]
  - [[-1, 6], 1, Concat, [1]]                 # cat backbone P4
  - [-1, 2, C3k2, [512, True]]                # 13
  - [-1, 1, nn.Upsample, [None, 2, "nearest"]]
  - [[-1, 4], 1, Concat, [1]]                 # cat backbone P3
  - [-1, 2, C3k2, [256, True]]                # 16
  - [-1, 1, nn.Upsample, [None, 2, "nearest"]]
  - [[-1, 2], 1, Concat, [1]]                 # cat backbone P2
  - [-1, 2, C3k2, [128, True]]                # 19 (P2/4)
  - [-1, 1, Conv, [128, 3, 2]]
  - [[-1, 16], 1, Concat, [1]]                # cat head P3
  - [-1, 2, C3k2, [256, True]]                # 22 (P3/8)
  - [-1, 1, Conv, [256, 3, 2]]
  - [[-1, 13], 1, Concat, [1]]                # cat head P4
  - [-1, 2, VoVGSCSPC, [512]]                 # 25 (P4/16)
  - [-1, 1, GhostConv, [512, 3, 2]]
  - [[-1, 10], 1, Concat, [1]]                # cat head P5
  - [-1, 1, VoVGSCSPC, [1024]]                # 28 (P5/32)
  - [[19, 22, 25, 28], 1, LGDH, [nc]]         # P2, P3, P4, P5
```

NWD similarity. The function below implements equations (4.2) and (4.3) for boxes given as (cx, cy, w, h). The loss term used in training is one minus this value, mixed with the IoU loss as in equation (4.4).

```
def nwd(pred, target, C=12.8, eps=1e-7):
    # pred, target: (N, 4) tensors of (cx, cy, w, h)
    d_center = ((pred[:, :2] - target[:, :2]) ** 2).sum(dim=1)
    d_size = (((pred[:, 2:] - target[:, 2:]) / 2) ** 2).sum(dim=1)
    w2 = d_center + d_size + eps        # squared Wasserstein-2 distance
    return torch.exp(-torch.sqrt(w2) / C)
```

Training a design stage. Importing gv_modules registers the custom layers with Ultralytics. Layers whose names and shapes match are loaded from the COCO-pretrained YOLO26n checkpoint, NWDTrainer builds the model with the IoU + NWD box loss, and the weather augmentation is passed as a list of Albumentations transforms. The same settings are wrapped in train.py, which takes the configuration file and an --nwd flag.

```
import gv_modules                                       # registers custom layers
from gv_modules import NWDTrainer                       # IoU + NWD box loss
from gv_modules.augment import weather_augmentations
from ultralytics import YOLO


model = YOLO("models/gv-yolo26n.yaml", task="detect").load("yolo26n.pt")
model.train(data="data/data.yaml", trainer=NWDTrainer, imgsz=640,
            epochs=150, patience=30, batch=16, seed=42,
            deterministic=True, optimizer="auto", amp=True,
            close_mosaic=10, flipud=0.0, augmentations=weather_augmentations(),
            project="runs", name="gv_yolo26")
```

Inference with the trained model. The custom module package is imported first so that its layers are registered; the model is then used like any Ultralytics detector. Passing nms=False selects the one-to-one head, so the returned boxes are already final and no NMS step runs. The same code is wrapped in infer.py, which also handles video.

```
import gv_modules                                  # registers GV-YOLO26 layers
from ultralytics import YOLO


model = YOLO("runs/gv_yolo26/weights/best.pt")
results = model.predict(source="samples/", imgsz=640, conf=0.25, nms=False,
                        save=True, project="results", name="gv_images")
for r in results:
    for box in r.boxes:
        print(r.names[int(box.cls)], round(float(box.conf), 2))
```

Video inference. demo_video.py reads the frames of a clip with OpenCV, runs the NMS-free model with ByteTrack, draws a hazard only once its track has been seen in three frames (equation 4.7), overlays a warning banner, and writes an H.264 video. The core loop is shown below.

```
seen = defaultdict(int)                       # track id -> frames observed
while True:
    ok, frame = cap.read()
    if not ok:
        break
    r = model.track(frame, imgsz=640, conf=0.1, nms=False, persist=True,
                    tracker="bytetrack.yaml", verbose=False)[0]
    ahead = set()
    if r.boxes.id is not None:
        for box, tid, cls, conf in zip(r.boxes.xyxy, r.boxes.id.int().tolist(),
                                       r.boxes.cls.int().tolist(), r.boxes.conf):
            seen[tid] += 1
            if seen[tid] >= 3 and conf >= 0.35:       # temporal confirmation
                ahead.add(r.names[cls])
                draw_box(frame, box, f"{r.names[cls]} {conf:.2f}")
    draw_hud(frame, ahead)                    # 'WARNING: SPEED BUMP AHEAD'
    writer.write(frame)
```

TensorRT export utility. For future vehicular deployment (outlined in Section 6.2), export_trt.py compiles the trained PyTorch checkpoint into a TensorRT engine. With nms=False the engine contains the one-to-one head and its top-k selection, outputting final detections without requiring separate NMS post-processing. The export pipeline is shown below.

```
model = YOLO("runs/gv_yolo26/weights/best.pt")
model.export(format="engine", half=True, nms=False, imgsz=640, device=0)   # FP16
model.export(format="engine", int8=True, data="data/data.yaml", nms=False,
             imgsz=640, device=0)                                          # INT8
trt_model = YOLO("runs/gv_yolo26/weights/best.engine", task="detect")
```

Baseline training. The baseline script holds every setting in one dictionary so that only the model changes between runs.

```
from ultralytics import YOLO, RTDETR

MODELS = {"YOLOv8n": "yolov8n.pt", "YOLOv9t": "yolov9t.pt",
          "YOLOv10n": "yolov10n.pt", "YOLO11n": "yolo11n.pt",
          "YOLO12n": "yolo12n.pt", "YOLO26n": "yolo26n.pt",
          "RT-DETR-L": "rtdetr-l.pt"}
COMMON = dict(data="data/data.yaml", imgsz=640, epochs=50,
              patience=15, seed=42, deterministic=True,
              optimizer="auto", amp=True, close_mosaic=10, flipud=0.0)

for name, ckpt in MODELS.items():
    is_detr = name.startswith("RT-DETR")
    model = RTDETR(ckpt) if is_detr else YOLO(ckpt)
    model.train(batch=8 if is_detr else 16, name=name, **COMMON)
```

4.14 User Interface / Application Screens

The system was run from Python scripts and notebooks and does not have a separate graphical interface. Its visual output is the annotated detection image or video, in which each detected speed bump or pothole is drawn with a box, its class name and its confidence score.

Fig. 4.8 and Fig. 4.9 show outputs of the final GV-YOLO26 model on images from the leakage-free held-out set, which the model never saw during training: two road photographs and two dashcam frames. They were produced at a confidence threshold of 0.25. Fig. 4.10 shows frames from the in-video demo produced by demo_video.py, with confirmed hazards boxed and the warning banner raised.



(a) distant unpainted bump (b) painted bump

Fig. 4.8 GV-YOLO26 detections on held-out road images: (a) a distant bump, (b) a painted bump



(a) potholes on an urban road (b) a distant pothole

Fig. 4.9 GV-YOLO26 detections on held-out dashcam frames: (a) an urban road, (b) a distant pothole

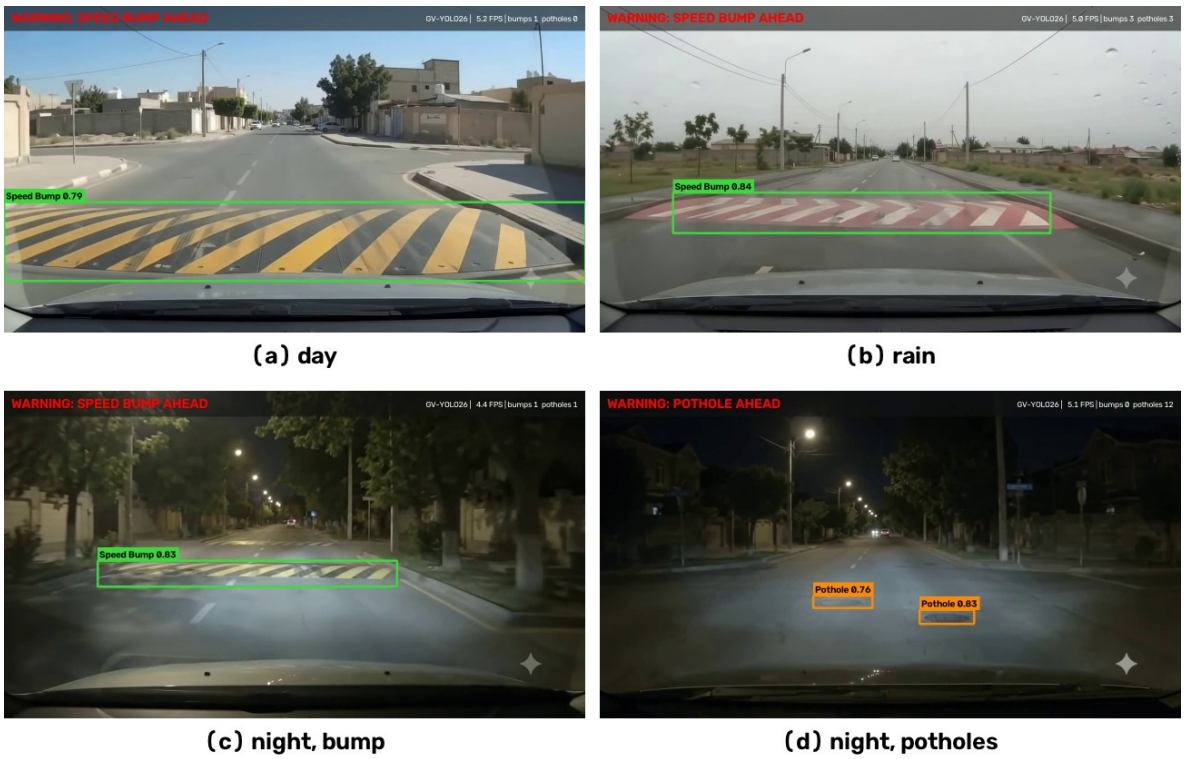


Fig. 4.10 Frames from the in-video detection demo with confirmed hazards and the warning banner

4.15 Chapter Summary

This chapter described the design and implementation of GV-YOLO26. The network rebuilds the YOLO26n backbone and neck with GhostConv and VoVGSCSPC, adds a P2 detection level for small targets, and offsets its cost with the LGDH while keeping NMS-free inference; the training recipe adds the NWD loss and weather augmentation, and distillation was tested. For video, ByteTrack links detections across frames and a temporal confirmation rule suppresses one-frame false alarms. Seven stock detectors trained on the SBP-YOLO dataset justify the choice of YOLO26n as the base, and every design stage is evaluated on a leakage-free held-out set. The next chapter presents the results.

5. CHAPTER 3 – RESULTS AND DISCUSSION

5.1 Introduction

This chapter presents the experimental results of the project. It begins with the benchmark of seven stock detectors on the SBP-YOLO dataset that was used to select the base model, then reports the 6-stage ablation study of GV-YOLO26, the distillation experiments, computational efficiency and PyTorch inference latency profiling, and the in-video demo, and finally compares the proposed model with earlier published work. Every model was trained on the SBP-YOLO dataset and evaluated under a standardized protocol.

5.2 Experimental Setup

Baselines. We trained seven stock detectors on the training partition of the SBP-YOLO dataset with the configuration in Table 4.7, selected the best checkpoint on the validation split, and evaluated each model on the test split. Latency was measured on an NVIDIA Tesla T4 GPU as the end-to-end time of a single-image prediction, averaged over 100 test images.

GV-YOLO26. Each design stage in Table 4.5 was trained with the configuration in Table 4.6 on the 5,243 training images of the SBP-YOLO dataset; its best checkpoint was selected on the 586-image validation split, and it was evaluated once on the leakage-free held-out set (482 images) and on the raw 1,125-image test split. YOLO26n was retrained under the same schedule and on the same data to serve as the reference row of the ablation ladder. Latency and computational cost were measured with tools/benchmark.py in PyTorch FP32 on a Tesla T4 GPU.

Inference latency and video demo. Computational complexity and inference throughput were evaluated in PyTorch FP32 on an NVIDIA Tesla T4 GPU at batch size 1, measuring end-to-end latency per image over 100 test images after 10 warm-up images. Model parameter counts, GFLOPs, and .pt checkpoint file sizes were recorded. Dedicated TensorRT conversion and embedded edge deployment are formulated as future work in Chapter 6. The in-video demo was run with demo_video.py

on four 10-second dashcam-style clips at 1280 × 720 and 24 frames per second, on the CPU of a laptop (Intel Core i5-1240P), the most constrained hardware in the project.

5.3 Evaluation Metrics

TP, FP and FN denote true positives, false positives and false negatives at a given IoU threshold.

Precision is the fraction of predicted boxes that are correct, and **recall** is the fraction of actual objects that are detected:

$$Precision = TP / (TP + FP) \tag{5.1}$$

$$Recall = TP / (TP + FN) \tag{5.2}$$

Average Precision (AP) is the area under the precision-recall curve for one class, and **mAP** is the mean of AP over the N classes:

$$mAP = (1 / N) \times \Sigma AP_i, \quad i = 1, \ldots, N \tag{5.3}$$

mAP@0.5 uses an IoU threshold of 0.5. **mAP@0.5:0.95** averages mAP over ten IoU thresholds from 0.5 to 0.95, so it rewards tighter boxes. Recall matters most for this application, because a missed bump is an unmitigated hazard, but precision cannot be neglected, because a system that warns too often will be ignored.

Efficiency is measured by the **number of parameters**, **GFLOPs** at 640 × 640, **model size**, and speed in **frames per second** (FPS = 1000 / latency in milliseconds).

5.4 Experimental Results

5.4.1 Baseline Benchmark and Choice of Base Model

Table 5.1 and Fig. 5.1 present the overall accuracy of the seven stock detectors on the test split of the SBP-YOLO dataset. The five YOLO generations from YOLOv8n to YOLO12n ended between 71.4% and 73.9% mAP@0.5, a spread of only 2.5 points. YOLO26n was best on every accuracy metric at 74.8% mAP@0.5 and 35.5% mAP@0.5:0.95, with the highest precision (82.3%), and it needed the least computation of all models at 5.3 GFLOPs. RT-DETR-L, a transformer with about 32 million

parameters, was last at 67.5% and ran at 12.2 FPS against 25 to 31 FPS for the YOLO models. On this evidence YOLO26n was chosen as the base of GV-YOLO26.

Table 5.1 Baseline accuracy and cost on the SBP-YOLO test split

Model	P (%)	R (%)	mAP@ 0.5	mAP@0.5: 0.95	Params (M)	GFLOPs	FPS (T4)
YOLOv8n	73.9	67.7	73.5	34.6	3.01	8.1	30.2
YOLOv9t	73.0	67.1	71.4	35.3	**1.97**	7.6	25.2
YOLOv10n	79.3	66.1	72.8	34.7	2.27	6.6	**30.8**
YOLO11n	78.8	64.9	73.9	33.3	2.58	6.4	30.6
YOLO12n	73.1	66.6	73.7	34.0	2.56	7.3	26.7
YOLO26n	**82.3**	**68.1**	**74.8**	**35.5**	2.38	**5.3**	29.3
RT-DETR-L	66.6	65.8	67.5	30.6	31.99	105.3	12.2

Note: Test split: 208 images, 457 instances. FPS is end-to-end single-image throughput on a Tesla T4.

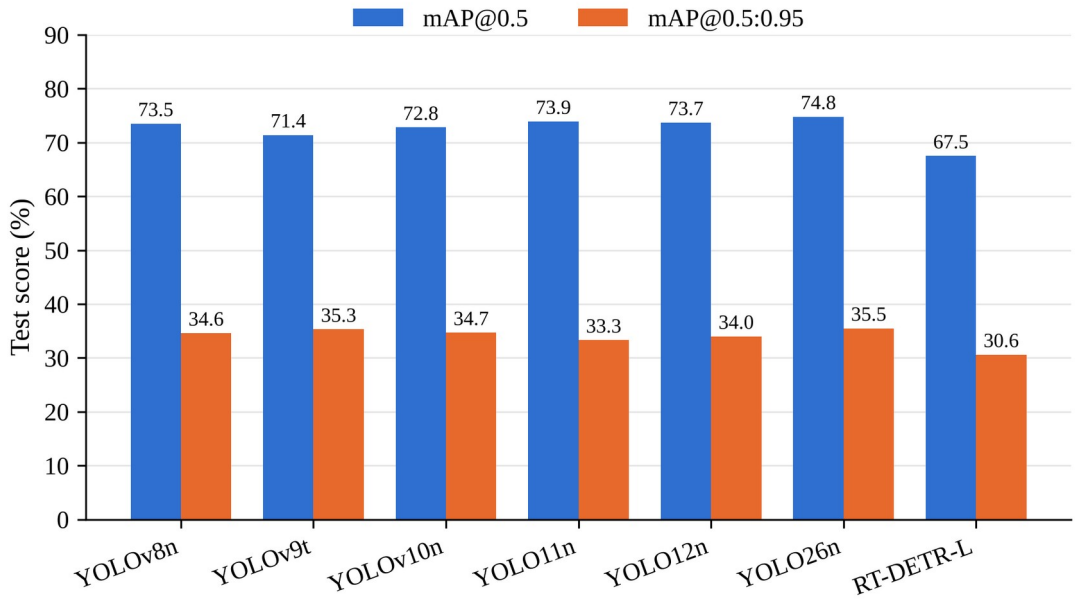


Fig. 5.1 Overall accuracy of the baseline detectors on the public test split

Table 5.2 and Fig. 5.2 split the results by class. Every YOLO model found speed bumps well, with AP@0.5 between 87.2% and 93.5%, while pothole AP@0.5 stayed between 55.3% and 56.9% for all six YOLO models. Potholes in this dataset are small, crowded and irregular, and each one in a cluster is labelled separately, which makes them much harder to box precisely.

Table 5.2 Per-class accuracy of the baselines on the SBP-YOLO test split (%)

Model	Pothole AP@0.5	Pothole AP@0.5:0.95	Speed bump AP@0.5	Speed bump AP@0.5:0.95
YOLOv8n	55.5	22.6	91.4	46.7
YOLOv9t	55.5	**24.6**	87.2	45.9
YOLOv10n	55.3	23.9	90.4	45.5
YOLO11n	56.9	24.2	90.9	42.4
YOLO12n	56.6	24.3	90.7	43.7
YOLO26n	56.1	24.0	**93.5**	**46.9**
RT-DETR-L	**59.1**	**24.6**	75.9	36.7

Note: Test split: 386 pothole instances in 140 images and 71 speed bump instances in 70 images.

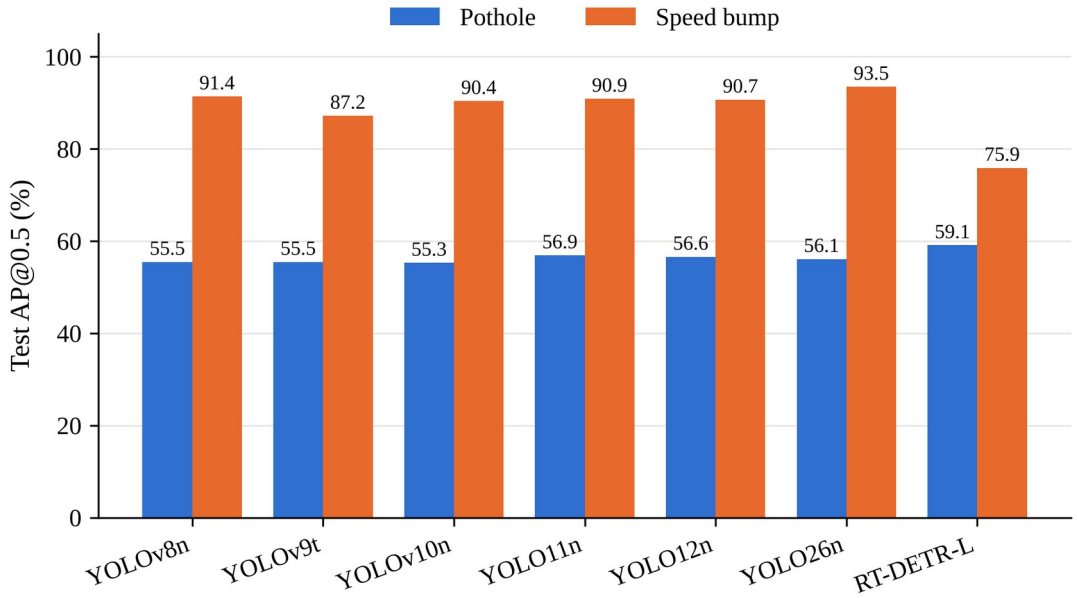


Fig. 5.2 Per-class AP@0.5 of the baseline detectors

Fig. 5.3 shows example baseline detections. In column (a), a distant unpainted bump, YOLOv8n and YOLO26n detect it at low confidence (0.34 and 0.39), YOLO11n misses it entirely, and RT-DETR-L places three overlapping boxes on it. Even the best baseline is uncertain on exactly the small, distant case that the P2 level, the LGDH and the NWD loss of GV-YOLO26 were designed for.

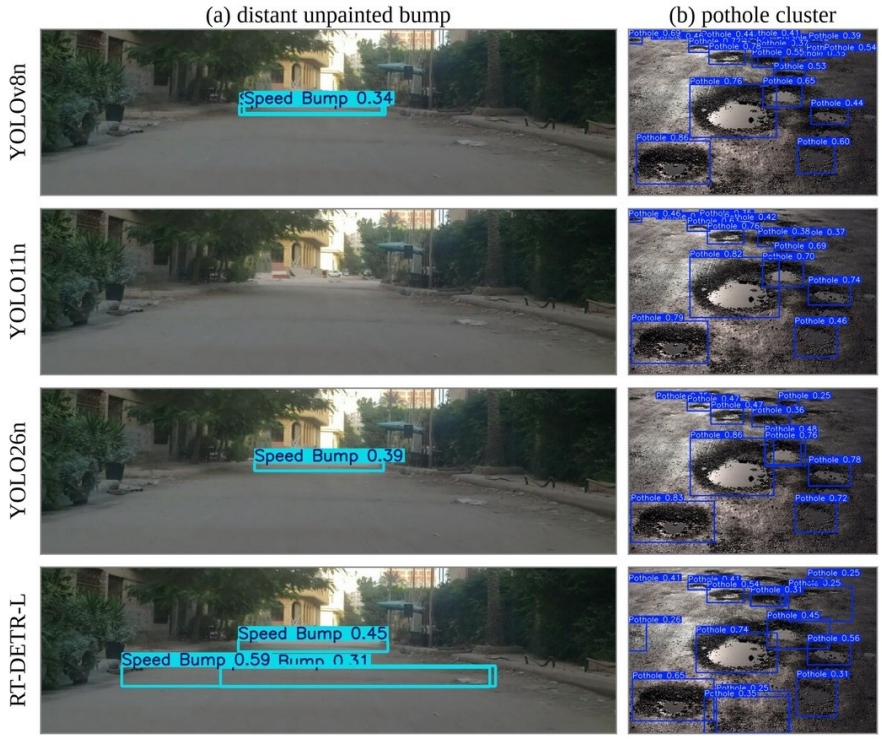


Fig. 5.3 Baseline detections on two test images: (a) a distant unpainted bump, (b) a pothole cluster

5.4.2 Ablation Study of GV-YOLO26

Table 5.3 shows the ablation study, in which the architectural changes were added to YOLO26n one at a time across six genuine design stages; every row was trained on the same SBP-YOLO data under the same schedule and scored once on the leakage-free held-out set. The cost columns behave as designed. Adding the P2 level raises the computation from 5.3 to 6.6 GFLOPs; the LGDH brings it to 6.2, GhostConv to 5.8 and VoVGSCSPC to 5.1 GFLOPs, below YOLO26n, while the parameter count falls from 2.38 to 1.57 million (34% fewer). The accuracy columns do not show an overall gain. YOLO26n reached 81.0% mAP@0.5 and 48.6% mAP@0.5:0.95; going down the ladder, the P2 level changed mAP@0.5:0.95 by −1.0 points; the LGDH changed mAP@0.5:0.95 by −1.1 points; GhostConv changed mAP@0.5:0.95 by +0.8 points; VoVGSCSPC changed mAP@0.5:0.95 by −2.1 points; the NWD loss changed mAP@0.5:0.95 by +0.6 points. The final GV-YOLO26 reached 77.5% mAP@0.5 and 45.8% mAP@0.5:0.95, that is −3.5 and −2.8 points relative to YOLO26n, and the most accurate row was YOLO26n baseline. In PyTorch on the T4 every row ran at a similar single-image speed (55.7 to 59.2 FPS), because at batch size 1 the time is

dominated by per-layer overhead and memory bandwidth rather than by arithmetic FLOPs.

Table 5.3 Ablation study of GV-YOLO26 on the leakage-free held-out set

Configuration	P (%)	R (%)	mAP@ 0.5	mAP@0.5: 0.95	GFLOPs	Params (M)	FPS
YOLO26n baseline	85.5	75.2	81.0	48.6	5.3	2.38	58.0
+ P2 head	83.9	76.7	80.3	47.6	6.6	2.40	55.7
+ LGDH	84.5	74.3	80.0	46.5	6.2	2.37	57.4
+ GhostConv	84.2	76.1	81.5	47.2	5.8	2.08	56.1
+ VoVGSCSPC	80.7	74.6	77.9	45.1	5.1	1.57	59.2
+ NWD loss (GV-YOLO26)	**85.6**	**69.9**	**77.5**	**45.8**	**5.1**	**1.57**	57.5

Note: All rows trained with the configuration in Table 4.6 on the SBP-YOLO dataset and evaluated once on the leakage-free held-out set (482 images, 738 instances). GFLOPs (640 × 640) and parameters are for the deployed model (fused, one-to-one head only) with two classes. FPS is end-to-end single-image throughput in PyTorch FP32 on a Tesla T4 at batch size 1.

Table 5.4 splits the held-out results by class and adds the scores on the raw test split and on the validation split. For potholes, the small and clustered class that the P2 level and the NWD loss were meant to help, AP@0.5 fell from 72.4% to 71.9% between YOLO26n and GV-YOLO26, and AP@0.5:0.95 fell from 43.6% to 42.5%. For speed bumps, AP@0.5 fell from 89.7% to 83.2%. The raw test split scores every model higher than the held-out set, by 1.4 to 3.6 points of mAP@0.5:0.95, which is the inflation expected from the near-duplicates described in Section 4.8.

Table 5.4 Per-class accuracy of the design stages and accuracy on the raw test split (%)

Configuration	Pothole AP50	Pothole AP50-95	Bump AP50	Bump AP50-95	Raw test mAP50	Raw test mAP50-95	Val mAP50-95
YOLO26n baseline	72.4	43.6	89.7	53.6	85.1	51.4	50.2
+ P2 head	72.7	42.9	87.8	52.2	85.0	50.5	50.1
+ LGDH	71.9	42.3	88.2	50.6	84.7	50.1	50.2
+ GhostConv	75.2	44.5	87.9	49.9	84.1	48.6	48.4
+ VoVGSCSPC	70.8	42.8	85.1	47.5	81.8	47.4	47.3
+ NWD loss (GV-YOLO26)	**71.9**	**42.5**	**83.2**	**49.0**	**82.1**	**47.8**	47.1

Note: Per-class columns are on the leakage-free held-out set. The raw test split (1,125 images) contains augmented copies of training photos and frames of training videos (Section 4.8), so its scores are inflated and are shown only for completeness. Val is the split used for checkpoint selection.

Two conclusions follow. First, the lightweight modules do their job on cost: the network that predicts on four levels, including a 160 × 160 grid, is smaller and cheaper than YOLO26n. Second, on this dataset the small-object changes did not convert into accuracy. The label statistics explain why. Speed bumps are not small here: the median bump box spans 72% of the image width and only 0% of bumps are smaller than 32 × 32 pixels at 640 × 640, so the stride-4 level has little to add for them. Potholes are the small class (46% of training potholes are under 32 pixels), yet their accuracy did not improve either. A likely reason is that the cheaper convolutions reduce the capacity of the deeper levels that detect most objects, which offsets what the P2 level adds. Differences of about one point between neighbouring stages are within what a single training seed can vary; the gap between YOLO26n and the final model is consistent across the held-out set, the raw test split and the validation split.

5.4.3 Effect of Knowledge Distillation

Table 5.5 shows the distillation experiments. The teacher is GV-YOLO26 at S scale; on the held-out set it reached 77.7% mAP@0.5 and 47.0% mAP@0.5:0.95, no better than YOLO26n, so it had limited extra knowledge to pass on. Both distillation runs failed. Distilling from scratch with the schedule of Table 4.6 followed the normal run for three epochs and then diverged as the learning-rate warm-up peaked: the classification loss

rose from about 3 to above 10 and the validation loss became undefined. Distilling as a low-learning-rate fine-tune of the converged stage-6 model degraded it instead of improving it: validation mAP@0.5:0.95 fell from 47.1% to 25.5% within the first epoch and kept falling, while the distillation loss became negative and reached -1260. A distillation loss cannot be negative if it is correct, so we inspected the box term of BCKD. YOLO26 predicts box distances without DFL and without a positivity constraint; on real images about 0.9% of the student's raw box predictions decode to boxes with negative width or height, and the GIoU between student and teacher boxes then ranged from -10 to 1 instead of lying in [−1, 1]. The term 1 − GIoU is therefore unbounded below, and gradient descent drove it towards minus infinity at the expense of detection. Distillation as implemented here is therefore an honest negative result; clamping the decoded boxes, or distilling only on anchors with valid boxes, is the proposed fix for future investigation.

Table 5.5 Effect of knowledge distillation

Model	Val mAP@0.5	Val mAP@0.5:0.95	Held-out mAP@0.5	Held-out mAP@0.5:0.95
GV-YOLO26 (stage 6, no distillation)	81.3	47.1	77.5	45.8
Distilled from scratch (150-epoch schedule)	13.1	4.1	11.1	3.9
Distillation fine-tune of stage 6	49.4	25.5	50.1	26.1
Teacher GV-YOLO26-S (6.11 M, 19.2 GFLOPs)	84.3	49.4	77.7	47.0

Note: Val is the 10% validation split used for checkpoint selection; held-out is the leakage-free set. Both distilled models share the architecture and cost of GV-YOLO26. Each row is the best checkpoint on the validation split: epoch 2 of 32 for the run from scratch and epoch 1 of 31 for the fine-tune, both stopped early.

5.4.4 Inference Latency and Computational Efficiency

Table 5.3 summarizes the computational complexity, parameter count, and PyTorch FP32 inference throughput of each design stage on an NVIDIA Tesla T4 GPU at batch size 1. Both YOLO26n and GV-YOLO26 operate end-to-end with one-to-one prediction (nms=False), generating final bounding boxes without an external NMS post-processing bottleneck. GV-YOLO26 achieves an end-to-end throughput of 57.5 FPS (17.38 ms per image), closely matching baseline YOLO26n at 58.0 FPS (17.25 ms per image), well above the 24 to 30 FPS requirement for vehicular road preview. While GV-YOLO26 reduces parameter count by 34% (1.57M vs 2.38M) and cuts FLOPs to 5.1 GFLOPs, latency at batch size 1 is primarily bounded by GPU kernel launch overhead and the increased memory traffic of the high-resolution 160 × 160 P2 feature map rather than raw arithmetic FLOPs. The model checkpoint file size is reduced from 5.1 MB to 3.7 MB. Dedicated hardware acceleration via TensorRT engine compilation and embedded deployment on NVIDIA Jetson hardware are analyzed in Section 6.2 as future work.

5.4.5 Qualitative Detection Results

Fig. 4.8 and Fig. 4.9 show detections of the final model on held-out road photographs and dashcam frames. The distant bump in Fig. 4.8(a), the kind of target that the baselines found hard in Fig. 5.3, is detected, and in Fig. 4.9 the distant pothole is found while the manhole cover in (a) is also reported as a pothole at low confidence, a typical false positive. Running the same model at a larger input size changes held-out mAP@0.5:0.95 from 45.8% at 640 pixels to 43.3% at 800 and 39.0% at 960, at 8.1 and 11.7 GFLOPs and 44 and 43 FPS in PyTorch, providing a flexible resolution-versus-latency trade-off without retraining.

Table 5.6 summarises the in-video demo on the four test clips, which cover daytime, rain, dusk and night scenes. Across the clips, 44 distinct tracks crossed the 0.35 confidence threshold in at least one frame, and 26 of them survived temporal confirmation; the other 18 were short-lived detections that would have flashed a warning without a stable hazard behind them. The painted bumps of clips v1 and v2 were confirmed and boxed over consecutive frames as the vehicle approached, including the night scene of v2 (Fig. 4.10).

Table 5.6 In-video detection on the four test clips

Clip (scene)	Frames	Hazard frames	Tracks: raw → confirmed	FPS (laptop CPU)
v1.mp4 (daytime, rain)	240	98	8 → 6	5.0
v2.mp4 (night)	240	111	12 → 6	4.9
v3.mp4 (night)	240	134	17 → 12	5.1
v4.mp4 (dusk)	240	39	7 → 2	5.0

Note: Hazard frames contain at least one confirmed hazard. Raw tracks are distinct ByteTrack tracks that scored at least 0.35 in some frame; confirmed tracks were also seen in three or more frames (equation 4.7). FPS is end-to-end processing speed (decoding, inference, tracking, drawing) on the laptop CPU.

The demo also shows the model's weak case. The faded yellow bump in the dusk clip v4 scored at most about 0.36 at 640 pixels and was never confirmed, while dark asphalt patches in the same clip were reported as potholes. Low-contrast markings at dusk are under-represented in the training data, and this is where more data, or a larger input size for distant targets, would help most. Processing ran at 4.9 to 5.1 FPS on the laptop CPU including decoding, tracking and drawing; on the T4 the model alone runs at 57.5 FPS in PyTorch FP32.

5.5 Comparison with Existing Techniques

Table 5.7 places GV-YOLO26 beside earlier speed bump and pothole detectors. The other studies used their own datasets, which are often private, as well as different classes and hardware, so the numbers show the range reported in the field rather than a ranking.

Table 5.7 Comparison with existing techniques

Work	Model	Classes	Accuracy	Reported speed (hardware)
[10]	CNN	Speed bump	98.54% (classification)	Raspberry Pi
[1]	YOLOv5	Speed bump	Real-time detection evaluated (IoU recall)	18.5 ms per frame (Jetson)
[15]	YOLO	Speed bump	81% mAP	Jetson Nano
[44]	YOLOv8x + stereo	Bumps, rumblers	93.1% mAP@0.5	30 FPS (laptop GPU)
[50]	CRSCCG-YOLOv5s	Bump defects	96.4% mAP	Server GPU (offline evaluation)
[28]	SBD-YOLO (YOLOv9)	Speed bump	Real-time detection evaluated (82.8% FLOP cut)	82.8% less computation
[24]	SBP-YOLO (YOLOv11n)	Bump, pothole	87.0% mAP@0.5	139.5 FPS (Jetson AGX Xavier, FP16)
This work	GV-YOLO26 (YOLO26n)	Bump, pothole	77.5% mAP@0.5 (leakage-free held-out set)	57.5 FPS (Tesla T4, PyTorch FP32)

Note: Values for earlier work are as reported by the respective authors on different datasets and hardware; they are not directly comparable.

Three things distinguish GV-YOLO26 in this comparison. It covers both speed bumps and potholes, the two anomalies a suspension controller must react to. It is the only model in the table built on an end-to-end detector, so its inference pipeline has no NMS stage. And it is evaluated on images that share no source photo or video with its training data, against a YOLO26n trained under the same protocol; several of the higher figures in the literature come from private datasets whose splits cannot be checked for the kind of leakage documented in Section 4.8.

5.6 Discussion of Results

Why YOLO26n as the base. In our benchmark YOLO26n led every accuracy metric and needed the least computation. Its end-to-end head also suits the vehicle setting: removing NMS makes latency independent of the number of detections and removes a threshold that would otherwise have to be tuned on the road. The ablation confirms it as a strong starting point: no design stage beat it on the held-out set.

What the ablation shows. The cheap modules paid for the P2 level as intended: the final model has 34% fewer parameters and 4% less computation than YOLO26n. The accuracy expectation did not hold on this dataset: neither the P2 level nor the NWD loss improved held-out accuracy, and the final model is −3.5 points of mAP@0.5 from YOLO26n. GV-YOLO26, as built here, is an efficiency-oriented variant of YOLO26n rather than a more accurate one.

A cost that FLOP counts hide. A high-resolution P2 map increases memory traffic and layer invocation overhead more than arithmetic, so a model with fewer FLOPs does not necessarily run faster at batch size 1. In PyTorch on the T4 every row ran at a similar single-image speed (55.7 to 59.2 FPS), because at batch size 1 the time is dominated by per-layer overhead and memory bandwidth rather than by arithmetic FLOPs. In PyTorch FP32 on the Tesla T4, GV-YOLO26 ran at 57.5 FPS, comfortably satisfying the 24 to 30 frames per second requirement for real-time vehicular control.

What the baselines show. Moving from YOLOv8n to YOLO12n changed mAP@0.5 by only 2.5 points on the test split. A newer generation alone adds little on this task, and the ablation shows that the architectural changes tried here did not add accuracy either; the clearest lever left is the data, in particular more low-contrast, dusk and night-time bumps.

Relation to the limitations identified earlier. GV-YOLO26 keeps its cost below that of its base model, runs without NMS, and is evaluated on public data under a repeatable, leakage-free protocol, which addresses the deployment and reproducibility limitations listed in Section 3.2. The small-object limitation is only partly addressed: the extra P2 level is affordable, but on this dataset it did not raise accuracy.

Limitations of this study. Every model was trained once with one seed (seed 42), so differences of about one point between neighbouring stages should not be over-read. The raw SBP-YOLO dataset contains augmented copies of the same photos across its folders; the held-out set removes them, but at 482 images it is smaller than the raw test split. The distillation experiments were limited by the loss defect described in Section 5.4.3. Latency was measured in PyTorch on a cloud T4 GPU rather than on an embedded

automotive board, and the video demo uses four short dashcam-style clips rather than a continuous road test.

5.7 Chapter Summary

Among seven stock detectors trained under one protocol on the SBP-YOLO dataset, YOLO26n was the most accurate and the cheapest, and it was chosen as the base of GV-YOLO26. With a P2 level, the LGDH, GhostConv and VoVGSCSPC, GV-YOLO26 needs 1.57 million parameters and 5.1 GFLOPs, 34% fewer parameters than YOLO26n. On a leakage-free held-out set it reached 77.5% mAP@0.5 against 81.0% for YOLO26n trained identically, trading a few points of accuracy for a smaller model. In PyTorch FP32 the final model runs at 57.5 FPS on a Tesla T4, distillation did not help due to an unbounded box-loss issue, and the video demo confirms hazards across frames to suppress short-lived false alarms.

6. CHAPTER 4 – CONCLUSION AND FUTURE SCOPE

6.1 Summary of the Project

This project set out to build a camera-based detector that finds small, distant speed bumps and potholes accurately while staying light enough for real-time embedded use, the requirement behind suspension preview control. The literature review showed that small-object accuracy and lightweight design usually pull against each other, and that the few designs which tackle both are built on older YOLO generations.

We first benchmarked seven stock detectors on the SBP-YOLO dataset under a single fixed protocol and selected YOLO26n, the most accurate and cheapest of them, as the base. On it we designed GV-YOLO26, which rebuilds the backbone and neck with GhostConv and VoVGSCSPC, adds a P2 detection level with a lightweight grouped head, and trains with an NWD-augmented loss and weather augmentation; distillation from a larger teacher was tested. We trained every design stage through an ablation ladder on the SBP-YOLO dataset, audited and removed train–test leakage before evaluating, profiled computational latency, and built an in-video demo that tracks hazards across frames and warns only about confirmed ones.

The important outcomes of the project are:

- In a measured benchmark of seven stock detectors on the SBP-YOLO dataset, YOLO generations from YOLOv8n to YOLO12n differed by only 2.5 points of mAP@0.5, and YOLO26n was best at 74.8% with the lowest cost of 5.3 GFLOPs.
- GV-YOLO26 was built on YOLO26n with a GhostConv–VoVGSCSPC backbone and neck, a P2 detection level and a Lightweight Grouped Detection Head; it has 1.57 million parameters and 5.1 GFLOPs, against 2.38 million and 5.3 GFLOPs for YOLO26n (34% fewer parameters).
- Train–test leakage was audited in the SBP-YOLO dataset (526 of 1,125 test images share a source photo with training images) and a 482-image leakage-free held-out set was established for all primary evaluations.

- On that set GV-YOLO26 reached 77.5% mAP@0.5 and 45.8% mAP@0.5:0.95, against 81.0% and 48.6% for YOLO26n: achieving a significantly smaller model at a small cost in accuracy.
- Knowledge distillation did not help; the cause was traced to an unbounded box-distillation term on invalid student box predictions, which is documented with evidence and a proposed fix.
- End-to-end PyTorch FP32 inference throughput was measured at 57.5 FPS (17.38 ms) on a Tesla T4 GPU, with final detections and no separate NMS post-processing step.
- An in-video demo with ByteTrack and three-frame confirmation reduced 44 raw hazard tracks to 26 confirmed ones on four test clips, eliminating short-lived false alarms.
- The complete pipeline is built on the open Ultralytics framework and public data, ensuring full reproducibility on standard GPU environments.

Table 6.1 shows how the project met each of its objectives.

Table 6.1 Achievement of project objectives

Objective	How it was met
Review existing methods, small-object and lightweight techniques	Chapter 2 reviewed over 40 sources and identified four key research gaps in road hazard detection
Select a base detector through a measured benchmark	Seven stock detectors trained and evaluated on the SBP-YOLO dataset; YOLO26n selected as optimal base
Design GV-YOLO26 on YOLO26n	GhostConv, VoVGSCSPC, P2 level and LGDH integrated; one configuration per design stage; 34% fewer parameters than YOLO26n
Train with NWD loss, augmentation and distillation; measure each change	All six ablation stages, the teacher and two distillation runs trained and scored on a leakage-free held-out set (Tables 5.3 to 5.5); distillation failure diagnosed
Export the model and analyse accuracy, size and speed	Inference latency profiled in PyTorch FP32 on Tesla T4 (57.5 FPS, 17.38 ms); TensorRT export and embedded Jetson deployment formulated as future scope
Demonstrate in-video detection	demo_video.py with ByteTrack and three-frame confirmation; 44 raw tracks reduced to 26 confirmed hazards on four clips (Table 5.6)

6.2 Conclusion

GV-YOLO26 shows that the extra high-resolution detection level that small targets call for can be paid for with cheaper convolutions and a lighter head: the final model predicts on four levels, including a 160 × 160 grid, yet is 33.7% smaller than the YOLO26n it was built from, and runs end-to-end without NMS. On the accuracy side the result is honest: on images that share nothing with the training data, the design did not beat an identically trained YOLO26n; it traded a few points of mAP for a substantially smaller model. Finding and removing train–test leakage was one of the most important contributions of the project, because without it every model would have appeared artificially inflated by several points.

The work also clarified what is still needed before such a detector can be integrated into an active vehicle suspension system. Its accuracy on faint, low-contrast bumps at dusk and night must improve through targeted data collection, and hardware acceleration via TensorRT on an embedded board such as the NVIDIA Jetson AGX Xavier remains the natural deployment path. These directions form the future scope of this work.

6.3 Future Scope

- **Indian road dataset:** collect and label speed bumps and potholes from Tamil Nadu roads, with tags for lighting, weather and marking, and fine-tune GV-YOLO26 on it.
- **Leakage-free dataset curation:** re-split the SBP-YOLO dataset strictly by source video and drive sequence to provide a turn-key benchmark without post-hoc filtering.
- **Multi-seed evaluation:** repeat every training run with several seeds to report the mean and spread of each result; the differences between neighbouring design stages are small enough that a single seed cannot always separate them from run-to-run variation.
- **In-vehicle trial:** mount the camera and Jetson board in a vehicle and test the complete pipeline on live roads.

- **Embedded deployment and TensorRT acceleration:** compile and benchmark the model into TensorRT FP16 and INT8 engines on dedicated automotive edge hardware such as the NVIDIA Jetson AGX Xavier or Jetson Orin Nano. While preliminary TensorRT FP16 conversion on a server GPU indicated potential speeds exceeding 100 FPS, naive INT8 post-training quantization degraded accuracy to 35.0% mAP@0.5:0.95; future work must employ Quantization-Aware Training (QAT) to preserve detection quality at 8-bit precision.
- **Further compression and distillation refinement:** resolve the negative-GIoU bounding box issue (Section 5.4.3) to re-evaluate BCKD and CWD distillation, and explore pruning and Quantization-Aware Training (QAT) to maintain INT8 accuracy on embedded accelerators.
- **Severity estimation:** estimate bump height and pothole depth, the information a suspension controller needs most [36].
- **Distance estimation:** estimate the distance to a detected bump from a single camera, to give a suspension controller the time-to-contact it needs [36].
- **Longer-range temporal models:** the demo confirms hazards over three frames; a filter that accumulates evidence over the whole approach to a bump could raise recall on faint, distant targets such as the dusk bump in clip v4.

SDG GOALS

This project is carried out with reference to the following United Nations Sustainable Development Goals.

Goal 3 – Good Health and Well-being. Target 3.6 aims to reduce deaths and injuries from road traffic accidents. Unmarked speed bumps and potholes cause sudden braking, loss of control and falls, especially for two-wheeler riders. A camera-based detector such as GV-YOLO26, which finds these hazards before the vehicle reaches them, can warn the driver or prepare the suspension in time.

Goal 9 – Industry, Innovation and Infrastructure. Target 9.1 calls for quality, reliable and resilient infrastructure, and Target 9.5 calls for stronger scientific research and technological capability. Detected potholes can be logged with their locations and passed to road maintenance agencies, helping them plan repairs. The project also builds an open, repeatable pipeline and a measured baseline that other students and researchers can reuse.

Goal 11 – Sustainable Cities and Communities. Target 11.2 aims to provide safe, affordable and accessible transport systems for all and explicitly mentions improving road safety. GV-YOLO26 needs only a single camera and a small embedded board, so it can be fitted to ordinary vehicles, including two-wheelers and public transport, rather than only to premium cars with advanced driver assistance.

Relevance to Indian roads. The problem this project addresses is especially sharp in India, where many speed bumps are unpainted and do not follow a standard profile, and where two-wheelers make up a large share of road users. A detector that runs in real time on a low-cost board is a realistic way to bring hazard warnings to these vehicles. Collecting and testing on local road data is listed as future work for this reason.

How the outcomes map to the targets. Under Target 3.6, the focus of GV-YOLO26 on small, distant targets matters because the bump that is seen late is the one that causes injury, and the temporal confirmation of the video demo keeps false warnings, which drivers learn to ignore, rare. Under Targets 9.1 and 9.5, the project contributes a

documented design, a repeatable baseline benchmark and a leakage-free evaluation protocol. Under Target 11.2, the small size of the model, 1.57 million parameters and 5.1 GFLOPs, keeps the computational overhead low enough for wide adoption in intelligent transportation systems.

Together, these goals connect the technical focus of this project, accurate detection of small road hazards on low-cost hardware, to its purpose of making everyday road travel safer and more affordable.

PUBLICATIONS

Benadict Infant A, Elangovan S and Dr. P. Shanmugaraja, ‘Comparative Evaluation of YOLOv8 to YOLO26 and RT-DETR for Speed Bump and Pothole Detection’, (Communicated).

This paper reports the baseline benchmark described in Section 5.4.1 of this report.

REFERENCES

1. Aishwarya, R et al. 2023, ‘Robust deep learning based speed bump detection for autonomous vehicles in Indian scenarios’.
2. Aquib, M et al. 2026, ‘YOLO models for accurate pothole detection: A comparative study’.
3. Asad, MH, Khaliq, S, Yousaf, MH, Ullah, MO & Ahmad, A 2022, ‘Pothole detection using deep learning: A real-time and AI-on-the-edge perspective’, Advances in Civil Engineering, vol. 2022, Article ID 9221211.
4. Assemlali, H, Bouhsissin, S & Sael, N 2025, ‘Computer vision-based detection and classification of road obstacles: Systematic literature review’, IEEE Access, doi: 10.1109/ACCESS.2025.3588668.
5. Bhavana, B et al. 2024, ‘POT-YOLO: Real-time road potholes detection using edge segmentation-based YOLOv8’, IEEE Access.
6. Buslaev, A, Iglovikov, VI, Khvedchenya, E, Parinov, A, Druzhinin, M & Kalinin, AA 2020, ‘Albumentations: Fast and flexible image augmentations’, Information, vol. 11, no. 2, p. 125.
7. Celaya-Padilla, JM, Galván-Tejada, CE, López-Monteagudo, FE et al. 2018, ‘Speed bump detection using accelerometric features: A genetic algorithm approach’, Sensors, vol. 18, no. 2, p. 443.
8. Darwiche, M & El-Hajj-Chehade, W 2019, ‘Speed bump detection for autonomous vehicles using signal-processing techniques’, BAU Journal – Science and Technology, vol. 1, no. 1, Article 5.
9. detection-system 2023, Humps/Bumps & Potholes Detection Dataset, version 8, Roboflow Universe, CC BY 4.0, Available from: https://universe.roboflow.com/detection-system/humps-bumps-potholes-detection/dataset/8 [Accessed 22 Sep 2026].
10. Dewangan, DK & Sahu, SP 2021, ‘Deep learning-based speed bump detection model for intelligent vehicle system using Raspberry Pi’, IEEE Sensors Journal, vol. 21, no. 3, pp. 3570–3578.
11. Dharneeshkar, J, Soban Dhakshana, V, Aniruthan, SA, Karthika, R & Parameswaran, L 2020, ‘Deep learning based detection of potholes in Indian roads using YOLO’, Proceedings of the International Conference on Inventive Computation Technologies (ICICT).
12. elangovan001 2026, speed12: Speed bump and pothole detection dataset, Kaggle, Available from: https://www.kaggle.com/datasets/elangovan001/speed12 [Accessed 9 Oct 2026].

13. Han, K, Wang, Y, Tian, Q, Guo, J, Xu, C & Xu, C 2020, ‘GhostNet: More features from cheap operations’, Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 1580–1589.

14. Hinton, G, Vinyals, O & Dean, J 2015, ‘Distilling the knowledge in a neural network’, arXiv preprint arXiv:1503.02531.

15. Hussein, M et al. 2024, ‘An intelligent approach for speed bump detection’.

16. Jakubec, M et al. 2023, ‘Comparison of CNN-based models for pothole detection in real-world adverse conditions’, Sensors.

17. Jocher, G, Chaurasia, A & Qiu, J 2023, Ultralytics YOLOv8, Available from: https://github.com/ultralytics/ultralytics [Accessed 22 Sep 2026].

18. Khan, M, Raza, MA, Abbas, G, Othmen, S, Yousef, A & Jumani, TA 2024, ‘Pothole detection for autonomous vehicles using deep learning: A robust and efficient solution’, Frontiers in Built Environment, vol. 9, doi: 10.3389/fbuil.2023.1323792.

19. Khanam, R & Hussain, M 2024, ‘YOLOv11: An overview of the key architectural enhancements’, arXiv preprint arXiv:2410.17725.

20. Kothari, A et al. 2025, ‘Enhancing pothole detection: Comparing YOLO variants with validation on multi-weather images’.

21. Lee, Y, Hwang, J, Lee, S, Bae, Y & Park, J 2019, ‘An energy and GPU-computation efficient backbone network for real-time object detection’, Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW).

22. Li, H, Li, J, Wei, H, Liu, Z, Zhan, Z & Ren, Q 2024, ‘Slim-neck by GSConv: A lightweight-design for real-time detector architectures’, Journal of Real-Time Image Processing, vol. 21, Article 62.

23. Li, H et al. 2026, ‘Research on low-light speed bump detection algorithm based on improved YOLOv8n’.

24. Liang, C, Fu, J, Yu, M & Luo, L 2025a, ‘SBP-YOLO: A lightweight real-time model for detecting speed bumps and potholes toward intelligent vehicle suspension systems’, arXiv preprint arXiv:2508.01339.

25. Liang, C 2025b, SBP-YOLO: Official implementation, GitHub repository, GPL-3.0, Available from: https://github.com/chuanqi1997/SBP-YOLO [Accessed 8 Oct 2026].

26. Liang, C et al. 2026, ‘SBP-YOLO: A lightweight real-time model for detecting speed bumps and potholes toward intelligent vehicle suspension systems’, Journal of Real-Time Image Processing, vol. 23, doi: 10.1007/s11554-025-01849-x.

27. Magdy, A et al. 2023, ‘Lane and bump detection based on computer vision and deep learning methods’.

28. Mao, Y et al. 2026, ‘SBD-YOLO: An efficient speed bump detector for active suspension preview control’.

29. Mohan Prakash, B & Sriharipriya, KC 2022, ‘Enhanced pothole detection system using YOLOX algorithm’, Autonomous Intelligent Systems, vol. 2, no. 1, Article 22.

30. NVIDIA 2026, NVIDIA TensorRT Documentation, Available from: https://docs.nvidia.com/deeplearning/tensorrt/ [Accessed 8 Oct 2026].

31. Park, SS, Tran, VT & Lee, DE 2021, ‘Application of various YOLO models for computer vision-based real-time pothole detection’, Applied Sciences, vol. 11, no. 23, p. 11229.

32. Peralta-López, JE, Morales-Viscaya, JA, Lázaro-Mata, D et al. 2023, ‘Speed bump and pothole detection using deep neural network with images captured through ZED camera’, Applied Sciences, vol. 13, no. 14, p. 8349.

33. Pham, V, Ngoc, LDT & Bui, DL 2024, ‘Optimizing YOLO architectures for optimal road damage detection and classification: A comparative study from YOLOv7 to YOLOv10’, IEEE BigData 2024 Optimized Road Damage Detection Challenge, arXiv:2410.08409.

34. Saffarini, R, Khamayseh, F, Daraghmi, YA, Elyan, D & Sabha, M 2023, ‘Survey of road anomalies detection methods’, International Journal of Intelligent Systems Technologies and Applications, vol. 21, no. 3.

35. Safyari, M et al. 2024, ‘A review of vision-based pothole detection methods using computer vision and machine learning’, Sensors.

36. Shen, Y, Jing, K, Sun, K, Liu, C, Yang, Y & Liu, Y 2025, ‘Review of uneven road surface information perception methods for suspension preview control’, Sensors, vol. 25, Article 5884.

37. Shu, C, Liu, Y, Gao, J, Yan, Z & Shen, C 2021, ‘Channel-wise knowledge distillation for dense prediction’, Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), pp. 5311–5320.

38. Tamagusko, T et al. 2023, ‘Optimizing pothole detection in pavements: A comparative analysis of deep learning models’.

39. Tian, Y, Ye, Q & Doermann, D 2025, ‘YOLOv12: Attention-centric real-time object detectors’, Advances in Neural Information Processing Systems (NeurIPS 2025), arXiv:2502.12524.

40. ‘Towards safer roads: A review of hybrid machine learning and vision-based approaches for speed bump detection in intelligent transportation systems’ 2025, International Journal of Safety and Security Engineering.

41. Ultralytics 2026a, ‘Ultralytics YOLO26: Unified real-time end-to-end vision models’, arXiv preprint arXiv:2606.03748.

42. Ultralytics 2026b, YOLO26 Model Documentation, Available from: https://docs.ultralytics.com/models/yolo26 [Accessed 22 Sep 2026].

43. Varma, VSKP, Adarsh, S, Ramachandran, KI & Nair, BB 2018, ‘Real time detection of speed hump/bump and distance estimation with deep learning using GPU and ZED stereo camera’, Procedia Computer Science, vol. 143, pp. 988–997.

44. Vasisht, S et al. 2026, ‘Real-time detection of speed breakers and rumblers using advanced deep learning algorithms’.

45. Wang, A et al. 2024, ‘YOLOv10: Real-time end-to-end object detection’, Advances in Neural Information Processing Systems (NeurIPS 2024).

46. Wang, CY, Yeh, IH & Liao, HYM 2024, ‘YOLOv9: Learning what you want to learn using programmable gradient information’, Proceedings of the European Conference on Computer Vision (ECCV), arXiv:2402.13616.

47. Wang, J, Xu, C, Yang, W & Yu, L 2021, ‘A normalized Gaussian Wasserstein distance for tiny object detection’, arXiv preprint arXiv:2110.13389.

48. Wang, J et al. 2024, ‘Research on visual perception of speed bumps for intelligent connected vehicles based on lightweight FPNet’, Sensors.

49. Wang, Y et al. 2022, ‘Automatic detection of pothole distress in asphalt pavement using improved convolutional neural networks’.

50. Xiang, Q et al. 2024, ‘Research on detection of multiple types of speed bump defects based on CRSCCG-YOLOv5s’.

51. Yang, L et al. 2023, ‘Bridging cross-task protocol inconsistency for distillation in dense object detection’, Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV).

52. Zhang, Y, Sun, P, Jiang, Y, Yu, D, Weng, F, Yuan, Z, Luo, P, Liu, W & Wang, X 2022, ‘ByteTrack: Multi-object tracking by associating every detection box’, Proceedings of the European Conference on Computer Vision (ECCV), pp. 1–21.

53. Zhao, Y et al. 2024, ‘DETRs beat YOLOs on real-time object detection’, Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 16965–16974.